



INSTALLATION AND USE CASES

Apache CloudStack PoC Guide



UPDATED

2025



Apache CloudStack
Proof-of-Concept Guide
Installation and Operation Instructions

Copyright © ShapeBlue. All rights reserved. No part of this document may be reproduced, distributed, or transmitted in any form or by any means without the prior written permission of ShapeBlue.

“Apache”, “CloudStack”, “Apache CloudStack”, the Apache CloudStack logo, the Apache CloudStack Cloud Monkey logo and the Apache feather logos are registered trademarks or trademarks of The Apache Software Foundation.



*London - Mountain View
Cape Town – Bangalore*

Table of Contents

INTRODUCTION	3
ABOUT THIS GUIDE	5
CLOUDSTACK INSTALL GUIDE	7
HARDWARE REQUIREMENTS	8
SOFT REQUIREMENTS	9
INFRASTRUCTURE DESIGN	9
MANAGEMENT SERVER SETUP OVERVIEW	10
PREPARING THE MANAGEMENT SERVER	11
INSTALLING CLOUDSTACK	12
PREPARING THE NFS SHARES	12
INSTALLING THE DATABASE SERVER	12
KVM HYPERVISOR SETUP	14
PREPARING THE OPERATING SYSTEM	14
INSTALL AND CONFIGURE THE CLOUDSTACK AGENT	15
INSTALLING AND CONFIGURING LIBVIRT	16
CONFIGURING THE NETWORKING	17
ADDING A CLOUDSTACK ZONE	18
DOMAIN HIERARCHY, USER ACCOUNTS AND RESOURCE LIMITS	33
OVERVIEW	34
TOPICS COVERED	34
ACCESSING THE CONTROL PANEL	34
CREATING A DOMAIN	35
LIMITING RESOURCES AT THE DOMAIN LEVEL	36
ADDING A DOMAIN ADMINISTRATOR ACCOUNT	37
ADDING A USER ACCOUNT AND SETTING LIMITS	40
MANAGING GUEST INSTANCES	42
OVERVIEW	43
TOPICS COVERED	43
ACCESSING THE CONTROL PANEL	43
REGISTERING A NEW TEMPLATE	44
CREATING A GUEST NETWORK AND MANAGING EGRESS RULES	46
REGISTERING A USERDATA	49
CREATING AN INSTANCE RUNNING A WEB SERVER	51
CONFIGURING FIREWALL AND PORT FORWARDING FOR PUBLIC WEB SERVER ACCESS	51
SCALING UP/DOWN GUEST INSTANCES	55
REMOVING AND RECOVERING INSTANCES	57
VIRTUAL PRIVATE CLOUD	61
OVERVIEW	62
TOPICS COVERED	62
VPC ARCHITECTURE OVERVIEW	62
CREATING A VPC	64
CREATING ACL LISTS	65
CREATING NETWORK TIERS	69
REGISTERING A MANAGED USERDATA SCRIPT FOR THE BACKEND INSTANCES	72
CREATING BACKEND INSTANCES	73

CREATING AND CONFIGURING THE INTERNAL LOAD BALANCER	75
REGISTERING A MANAGED USERDATA SCRIPT FOR THE FRONTEND INSTANCES.....	78
CREATING FRONTEND INSTANCES	81
CREATING AND CONFIGURING THE EXTERNAL LOAD BALANCER FOR FRONTEND TIER	82
ACCESSING THE SERVICE WITHIN THE VPC.....	83
AUTOSCALE INSTANCE GROUP	84
OVERVIEW	85
TOPICS COVERED	85
ACCESSING THE CONTROL PANEL.....	85
CREATING A GUEST NETWORK AND MANAGING EGRESS RULES.....	86
CREATING A LOAD BALANCER WITH AUTOSCALING.....	88
REGISTERING A MANAGED USERDATA SCRIPT FOR THE AUTOSCALING	90
CREATING AN AUTOSCALING INSTANCE GROUP	91
SIMULATING AUTOSCALING USING APACHE BENCHMARK	97
REFERENCES AND RESOURCES	102

Introduction



Apache CloudStack is the leading open-source cloud orchestration platform, used by many of the world's largest public and private clouds. It is a multi-hypervisor, multi-tenant, high-availability Infrastructure as a Service (IaaS) cloud management platform.

CloudStack provides a cloud orchestration layer, automating the creation, provisioning, and configuration of IaaS components (such as virtual servers). It transforms existing virtual infrastructure into a cloud-based IaaS platform. By leveraging existing infrastructure, CloudStack significantly reduces the cost and time required for organizations to build a multi-tenant IaaS platform.

One of the platform's key advantages is its simplicity and ease of use, even in large-scale environments. With CloudStack, you can seamlessly orchestrate public, on-premises, and hybrid cloud environments without the need for a large operations team to manage them over time.

As more organizations build on-premises clouds or enter the service provider market with public clouds, the demand for the right set of tools to build, manage, and scale IaaS platforms continues to grow. However, choosing the appropriate technology stack can be challenging. Several factors must be considered, including future growth, team size, budget, project timelines, previous experience, available hardware, and existing infrastructure.



About this Guide



This Lab Guide provides detailed operational instructions designed to assist in the PoC evaluation process. By following our PoC guide, you will ensure the creation of a fully functional and thoroughly tested cloud management system. Additionally, you'll gain a clear understanding of what to expect from the technology and how to configure everything in a timely and efficient manner.

By the end of this PoC, you will have a highly available, reliable, and flexible CloudStack-powered cloud. You will gain the confidence to set up and manage a CloudStack IaaS, ensuring the smooth implementation of the cloud orchestration layer into your infrastructure.

Let's get started!

Conventions

The following conventions are used to highlight important areas and required inputs:



Highlight a section of interest.

Highlighted text that is related to an important area in the GUI or Component/Feature.



Highlight a button/item from the list in the GUI that requires direct interaction.

1

Numerical sequence when many steps are illustrated by a single screenshot.

Highlighted text which can be copied and pasted directly into the UI.

Highlight text which can be copied and pasted directly into the command prompt.

Text highlighted from command prompt output.

Note

Highlights an observation that provides additional information.

Warning

Highlights a warning that provides additional information.

CloudStack Install Guide



Warning

This is a basic Apache CloudStack setup intended solely for Proof-of-Concept purposes.

Hardware requirements

To establish a functional architecture for Apache CloudStack evaluation, the following hardware is the minimum requirement.

Storage

Primary and Secondary storage created as NFS exports on the CloudStack Management server

500GB of RAID based storage on CloudStack Management server

Hosts

Item	Quantity/Description
No. hosts (per cluster required)	3
Clusters/Pods (1 cluster per pod)	1
No. cores (per host)	8
Memory (per host)	32-64GB
Local storage	Disk to support Hypervisor/OS
Network Interfaces	2 Ethernet cards
Network throughput	1Gb/s

Management Server

Item	Quantity/Description
No. cores	8
Memory	16GB
Local storage	150GB for OS + 500GB for primary and secondary storage of RAID based storage
Network Interfaces	1 Ethernet card
Network throughput	1Gb/s

Networking

Item	Quantity/Description
No. Switches	1
VLAN	802.1q support required for advanced network zones.
No. ports	Enough ports to connect 2 interfaces on each host considering 1 port for public/guest networks and 1 port for storage/management (+) 1 interface for storage/management network for the CloudStack management server.
Throughput	1 Gb/s



Soft Requirements

Note

Below are the IP addresses, VLANs and hostnames used in this example cloud setup, but keep in mind that your setup may use different values. Adjust these values as needed for your environment to ensure connectivity and proper configuration.

IP Address / VLAN Space

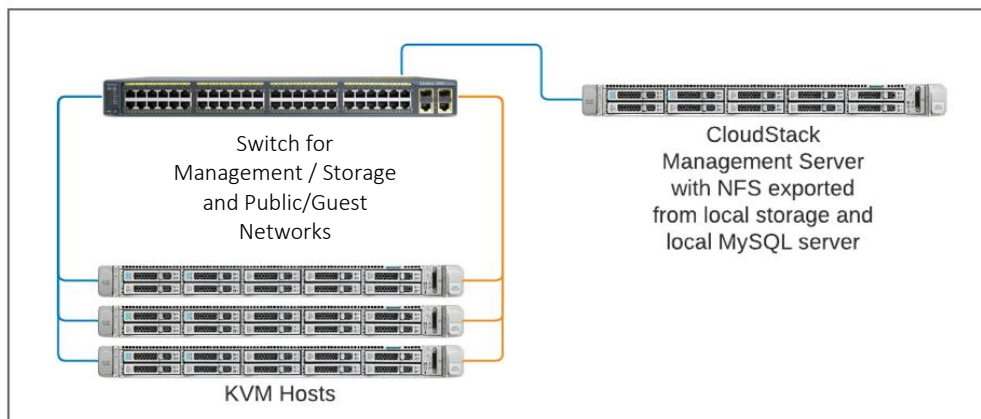
Item	Quantity/Description	Network IP Address	VLAN ID
Public Network	10 public addresses – /24 network IP range - RFC 1918 addresses routable within POC environment	10.0.48.0/24	48
Guest Network	20x VLANs dedicated to CloudStack use.		101-120
Management and Storage Network	40x RFC 1918 addresses	10.0.32.0/24	49

Hostname and IP addresses

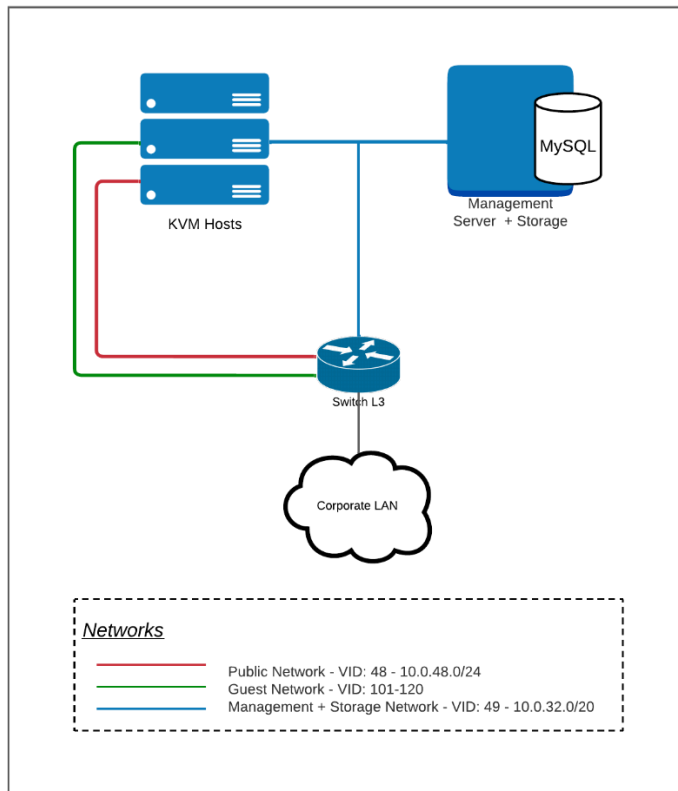
Host	hostname	IP Address	Netmask
CloudStack Management Server	mgmt.local	10.0.32.100	255.255.240.0
KVM Host 01	host01.local	10.0.33.1	255.255.240.0
KVM Host 02	host02.local	10.0.33.2	255.255.240.0
KVM Host 03	host03.local	10.0.33.3	255.255.240.0
Reserved System VM IP Address Range	-	10.0.34.1 – 10.0.34.40	255.255.240.0

Infrastructure Design

Physical Layout



Logical Layout



Management Server Setup Overview

Warning

Ensure you have a server that meets the hardware requirements before installing the management server.

Warning

In this guide, Alma Linux 8 will be used for both management servers and hosts. Ensure you have sufficient space to store the OS and the secondary storage mount point.

Warning

Before proceeding, ensure that the latest updates have been applied to your hosts.

Preparing the Management Server

1. Log in to the management server's OS as the root user.
2. Edit the `/etc/hosts` file and add the following lines. Alternatively, you can add these entries to your internal DNS server if preferred. Keep in mind that your setup may use different values. Adjust these values as needed for your environment.

```
10.0.32.10  mgmt      mgmt.local  secondary-storage.local primary-storage.local
10.0.33.1   host01    host01.local
10.0.33.2   host02    host02.local
10.0.33.3   host03    host03.local
```

3. Now, verify the fully qualified hostname.

```
hostname -f
```

Note

This should return a fully qualified hostname as `mgmt.local`.

4. Enable NTP for time synchronization.

```
dnf -y install chrony
systemctl enable --now chronyd
```

Note

A NTP daemon is required to synchronize the clocks of the servers in your cloud.

5. Set the SELINUX variable to `permissive` in `/etc/selinux/config` file. This ensures that the permissive setting is retained after a system reboot.

```
vi /etc/selinux/config
```

Change the following line:

```
SELINUX=enforcing
```

To:

```
SELINUX=permissive
```

6. Set SELinux to permissive immediately, without requiring a system reboot, by running the following command:

```
setenforce permissive
```

7. For a proof-of-concept propose, firewalld or iptables will not be necessary. To disable it, run the following command:

```
systemctl stop firewalld
systemctl disable firewalld
```

Installing CloudStack

1. Add the CloudStack repository by creating the `/etc/yum.repos.d/cloudstack.repo` file and inserting the following information.

```
[cloudstack]
name=cloudstack
baseurl= https://download.cloudstack.org/el/8/4.19/
enabled=1
gpgcheck=1
gpgkey=https://download.cloudstack.org/RPM-GPG-KEY
```

2. Now, install the CloudStack Management by running the following command:

```
dnf -y install cloudstack-management
```

Warning

CloudStack 4.19 requires Java 11 JRE. Installing CloudStack packages will automatically install Java 11, but it's important to explicitly confirm that the Java 11 is the selected version (especially if a previous version of Java is installed) with the command `alternatives --config java` after installing Apache CloudStack packages.

Note

Apache CloudStack requires space for Primary and Secondary Storage (refer to the CloudStack Design). Both storage types can be configured using NFS shares. This section explains how to set up the NFS shares before adding the storage to CloudStack.

Preparing the NFS Shares

1. Install the `nfs-utils` package:

```
dnf -y install nfs-utils
```

2. Create a NFS share for both primary and secondary storage by running the following command:

```
mkdir -p /export/primary
mkdir -p /export/secondary
```

3. To configure the new directories as NFS exports, edit `/etc/exports` file.

```
vi /etc/exports
```

Insert the following line:

```
/export *(rw,async,no_root_squash,no_subtree_check)
```

4. Now, export the `/export` directory by running the following command:

```
exportfs -a
```

Installing the Database Server

Note

We'll begin by installing MySQL and configuring specific options to ensure optimal performance with CloudStack. The MySQL 8 server will be installed from the AlmaLinux 8 AppStream repository, which needs to be enabled in the operating system.

1. Install `mysql-server` by running the following command:



```
dnf -y install mysql-server
systemctl enable mysqld
```

2. Edit the `/etc/my.cnf.d/mysql-server.cnf` configuration file.

```
vi /etc/my.cnf.d/mysql-server.cnf
```

Insert the following lines in the `[mysqld]` section:

```
server_id=1
innodb_rollback_on_timeout=1
innodb_lock_wait_timeout=600
max_connections=350
log-bin=mysql-bin
binlog-format = 'ROW'
```

3. Now, start MySQL server to apply the new configuration:

```
systemctl start mysqld
```

4. Run the follows command to set up the CloudStack database:

```
cloudstack-setup-databases cloud:password@localhost --deploy-as=root
```

It will configure the database with the following information.

Database	cloud
User	cloud
Password	password

5. Now, run the following commands to set up and start the Management Server:

```
cloudstack-setup-management
systemctl enable cloudstack-management
systemctl start cloudstack-management
```

Note

The Management Server should now be up and running.

KVM Hypervisor Setup

Note

To install the KVM hosts, ensure you have hosts that meet the hardware requirements.

Note

Before continuing, ensure that the latest updates have been applied to the hosts.

Warning

Repeat all of the following steps on each host.

The procedure for installing the Hosts include the following steps:

- Prepare the Operating System
- Install and configure libvirt
- Configure Security Policies (SELinux)
- Install and configure the Agent

Preparing the Operating System

1. Log in to the Host as root.
2. Edit the `/etc/hosts` file and add the following lines. Alternatively, you can add these entries to your internal DNS server if preferred.

```
10.0.32.10  mgmt      mgmt.local  secondary-storage.local primary-storage.local
10.0.33.1   host01    host01.local
10.0.33.2   host02    host02.local
10.0.33.3   host03    host03.local
```

3. Now, verify the fully qualified hostname.

```
hostname -f
```

Note

This should return a fully qualified hostname as `host0X.local`.

4. Enable NTP for time synchronization.

```
dnf -y install chrony
systemctl enable --now chronyd
```

Note

A NTP daemon is required to synchronize the clocks of the servers in your cloud.



- Set the SELINUX variable to *permissive* in */etc/selinux/config* file. This ensures that the permissive setting is retained after a system reboot.

```
vi /etc/selinux/config
```

Change the following line:

```
SELINUX=enforcing
```

To:

```
SELINUX=permissive
```

- Set SELinux to permissive immediately, without requiring a system reboot, by running the following command:

```
setenforce permissive
```

- For a proof-of-concept propose, firewalld or iptables will not be necessary. To disable it, run the following command:

```
systemctl stop firewalld
systemctl disable firewalld
```

Install and Configure the CloudStack Agent

- Add the CloudStack repository by creating the */etc/yum.repos.d/cloudstack.repo* file and inserting the following information.

```
[cloudstack]
name=cloudstack
baseurl= https://download.cloudstack.org/el/8/4.19/
enabled=1
gpgcheck=1
gpgkey=https://download.cloudstack.org/RPM-GPG-KEY
```

- Now, install the CloudStack Agent by running the following command:

```
dnf -y install cloudstack-agent
```

Warning

CloudStack 4.19 requires Java 11 JRE. Installing CloudStack packages will automatically install Java 11, but it's important to explicitly confirm that the Java 11 is the selected version (especially if a previous version of Java is installed) with the command *alternatives --config java* after installing Apache CloudStack packages.

Installing and Configuring Libvirt

Note

CloudStack uses *libvirt* to manage virtual machines in KVM, making it crucial that *libvirt* is configured correctly. As a dependency of *cloudstack-agent*, *libvirt* should already be installed.

Note

For live migration to function, *libvirt* must be configured to listen for unsecured TCP connections. Additionally, *libvirt's* attempt to use Multicast DNS advertising needs to be disabled. Both settings can be adjusted in */etc/libvirt/libvirtd.conf* file.

1. Open the */etc/libvirt/libvirtd.conf* file and set the following parameters:

```
listen_tls = 0
listen_tcp = 1
tcp_port = 16509
auth_tcp = "none"
mdns_adv = 0
```

Note

Enabling *listen_tcp* in *libvirtd.conf* is not sufficient; the parameters in */etc/sysconfig/libvirtd* also need to be modified.

2. Since Alma Linux 8 uses *systemd* socket activation, which effectively breaks *libvirtd --listen*, disable it by running the following command:

```
systemctl mask libvirtd.socket libvirtd-ro.socket libvirtd-admin.socket libvirtd-tls.socket libvirtd-tcp.socket
```

3. Edit */etc/sysconfig/libvirtd* file and uncomment the following line:

```
LIBVIRTD_ARGS=-l
```

4. Now, restart *libvirtd* to apply the changes by running the following command:

```
systemctl restart libvirtd
```

Configuring the Networking

Warning

This is a crucial section. Make sure you read this thoroughly.

Note

CloudStack uses network bridges in conjunction with KVM to connect guest Instances to each other and to the outside world. These bridges are also used to connect the System VMs to your infrastructure. By default, the bridges are named `cloudbr0` and `cloudbr1`.

Ensure that the interface names used for configuring the bridges follow one of the following patterns: `eth*`, `bond*`, `team*`, `vlan*`, `em*`, `p*p*`, `ens*`, `eno*`, `enp*`, or `enx*`. Otherwise, the KVM agent will not be able to configure the bridges properly.

Warning

Consistency in configuration across all hypervisors is essential.

Note

In Advanced Networking mode, it is common to have at least two physical interfaces per hypervisor host. We will use the `eth0` interface linked to the `cloudbr0` bridge, utilizing the untagged (native) VLAN for hypervisor management. Additionally, the second interface will be configured with the `cloudbr1` bridge for public and guest traffic. No VLANs are applied manually at this stage, as CloudStack will assign VLANs as needed during actual use.

Note

We advise against running these `nmcli` commands over an SSH session, as modifying the network connection on `eth0` will disrupt the SSH connection. Instead, execute these commands from the console directly. Alternatively, you may combine all the commands into a single bash script and run the script via SSH to avoid breaking the connection during the network configuration process.

1. To create an “empty” bridge and add `eth0` to the bridge, use the following commands:

```
nmcli connection add type bridge con-name cloudbr0 ifname cloudbr0
nmcli connection modify eth0 master cloudbr0
nmcli connection up eth0
```

2. Next, configure `cloudbr0` to set a static IP address along with the gateway:

```
nmcli connection modify cloudbr0 ipv4.addresses '10.0.33.x/20' ipv4.gateway '10.0.32.1' \
ipv4.dns '8.8.8.8' ipv4.method ipv4.method manual
nmcli connection up cloudbr0
```

3. To create the `cloudbr1` bridge as a plain bridge and add `eth1` to it, without an IP address or VLAN configuration, use the following commands:

```
nmcli connection add type bridge con-name cloudbr1 ifname cloudbr1
nmcli connection modify eth1 master cloudbr1
nmcli connection up eth1
```

4. Next, bring up the `cloudbr1` bridge:

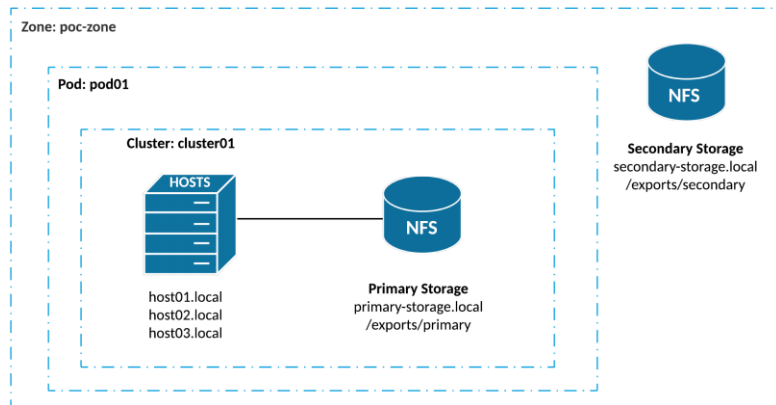
```
nmcli connection up cloudbr1
```



Adding a CloudStack Zone

Note

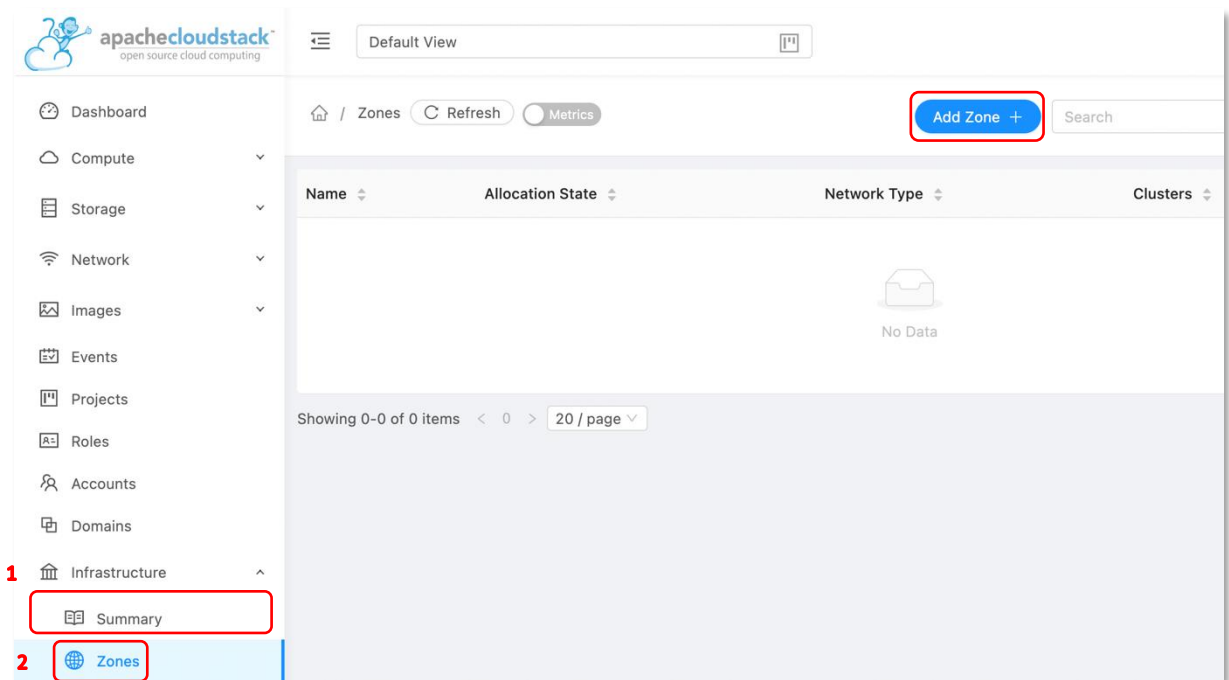
Next, each of the components represented below will be created.



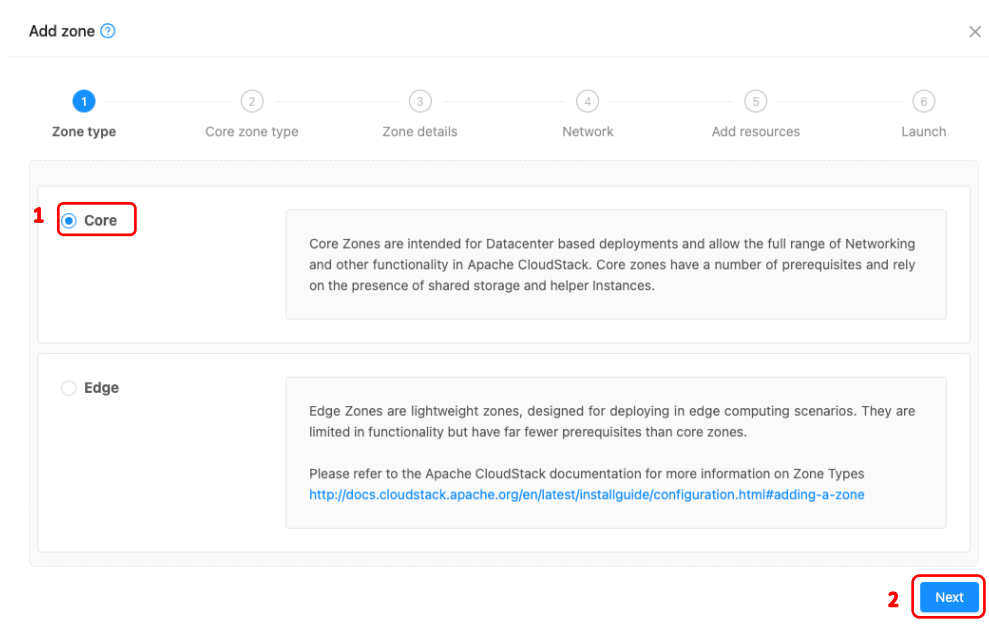
1. To access the CloudStack UI, open the following URL in your web browser:
<http://10.0.32.10:8080/client>
2. Login using the following credentials, then click *Login*.
 - Username: *admin*
 - Password: *password*
 - Domain: Blank



3. In the left of navigation pane, click *Infrastructure* > *Zones*, then click *Add Zone*.



4. In the wizard, select *Core*, click *Next* and then, on the next page, select *Advanced* to create an Advanced Zone and click *Next* again.



Add zone ⓘ

Zone type **Core zone type** Zone details Network Add resources Launch

1 ☒ **Advanced**

☐ Security groups

☐ Basic

Previous 2 **Next**

Note

For more info about CloudStack Advanced Zone, refer to this link:

http://docs.cloudstack.apache.org/en/latest/adminguide/networking/advanced_zone_config.html.

5. Fill in the informations required for *Zone Details* as follows and then click *Next*.

Name:	<i>poc-zone</i>
IPv4 DNS1:	<i>8.8.8.8</i>
Internal DNS 1:	<i>8.8.4.4</i>
Hypervisor:	<i>KVM</i>
Default guest CIDR for Isolated Networks:	<i>10.1.1.0/24</i>

Add zone ? ✕

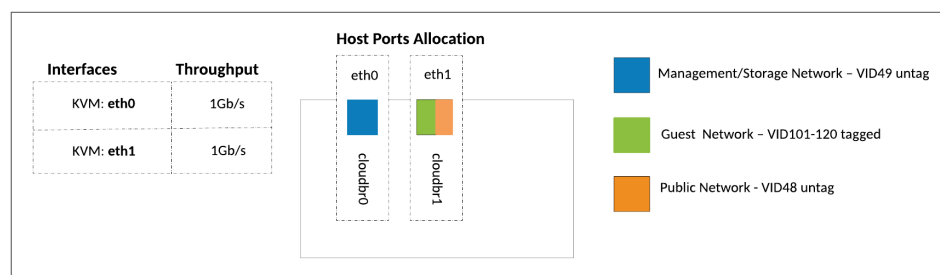
✓ ✓ 3 4 5 6
 Zone type Core zone type **Zone details** Network Add resources Launch

A zone is the largest organizational unit in CloudStack, and it typically corresponds to a single datacenter. Zones provide physical isolation and redundancy. A zone consists of one or more pods (each of which contains hosts and primary storage servers) and a secondary storage server which is shared by all pods in the zone.

* Name: poc-zone ✓
 * IPv4 DNS1: 8.8.8.8 ✓
 IPv4 DNS2: ✓
 IPv6 DNS1: ✓
 IPv6 DNS2: ✓
 * Internal DNS 1: 8.8.4.4 ✓
 Internal DNS 2: ✓
 * Hypervisor: KVM ✓
 Default network domain for Isolated networks:
 Default guest CIDR for Isolated Networks: 10.1.1.0/24
 Dedicated: ☐
 Enable local storage for User Instances: ☐
 Enable local storage for system VMs: ☐

Previous Next

6. Next, configure traffic types for the hosts' physical networks. Click [Add Physical Network](#) to create a new physical network and configure the [Traffic Types](#) as follows:



Add zone ⓘ

Zone type Core zone type Zone details **Network** Add resources Launch

Physical Network Public traffic Pod Guest traffic

When adding a zone, you need to set up one or more physical networks. Each physical network can carry one or more types of traffic, with certain restrictions on how they may be combined. Add or remove one or more traffic types onto each physical network.

Network name	Isolation method	Traffic types	Tags
Physical Network 1	VLAN	GUEST () MANAGEMENT () PUBLIC () + Add traffic	

Add physical Network

Previous Next

- Remove *Guest* and *Public* traffics from *Physical Network 1* clicking on the *garbage bin* button for both.

Add zone ⓘ

Zone type Core zone type Zone details **Network** Add resources Launch

Physical Network Public traffic Pod Guest traffic

When adding a zone, you need to set up one or more physical networks. Each physical network can carry one or more types of traffic, with certain restrictions on how they may be combined. Add or remove one or more traffic types onto each physical network.

Network name	Isolation method	Traffic types	Tags
Physical Network 1	VLAN	GUEST () MANAGEMENT () PUBLIC () + Add traffic	
Physical Network 2	VLAN	+ Add traffic	

Add physical Network

Previous Next

8. Add *Guest* and *Public* traffic in the *Physical Network 2* by clicking on *Add Traffic* button and then, click *Add* button:

Add zone ? ×

Zone type ✓ Core zone type ✓ Zone details ✓ **Network** 4 Add resources 5 Launch 6

Physical Network ● Pod ●

When adding a zone, you need to set up one or more physical networks. Each physical network can carry one or more types of traffic, with certain restrictions on how they may be combined. Add or remove one or more traffic types onto each physical network.

Network name	Isolation method	Traffic types	Tags	
Physical Network 1	VLAN	MANAGEMENT () ✎ 🗑 + Add traffic		
Physical Network 2	VLAN	+ Add traffic		🗑
Add physical Network				

Previous Next

Add zone ? ×

Zone type ✓ Core zone type ✓ Zone details ✓ **Network** 4 Add resources 5 Launch 6

Physical Network ● Pod ●

When adding a zone, you need to set up one or more physical networks. Each physical network can carry one or more types of traffic, with certain restrictions on how they may be combined. Add or remove one or more traffic types onto each physical network.

Network name	Isolation method	Traffic types	Tags	
Physical Network 1	VLAN	MANAGEMENT () ✎ 🗑 + Add traffic		
Physical Network 2	VLAN	PUBLIC 🗑 + ✖ STORAGE GUEST PUBLIC		🗑
Add physical Network				

Previous Next

Add zone ?

Zone type Core zone type Zone details **Network** Add resources Launch

Physical Network Public traffic Pod

When adding a zone, you need to set up one or more physical networks. Each physical network can carry one or more types of traffic, with certain restrictions on how they may be combined. Add or remove one or more traffic types onto each physical network.

Network name	Isolation method	Traffic types	Tags
Physical Network 1	VLAN	MANAGEMENT () + Add traffic	
Physical Network 2	VLAN	PUBLIC () GUEST	

Add physical Network

Previous Next

9. Click *Edit* (the pencil button) for *each traffic type* and set the traffic label as follows. Once all traffic labels are defined, click *Next*:

Name	Traffic Type	Traffic label
Physical Network 1	- Management	cloudbr0
Physical Network 2	- Public	cloudbr1
	- Guest	

Add zone ⓘ

Zone type ✓ Core zone type ✓ Zone details ✓ **Network** 4 Add resources 5 Launch 6

Physical Network Public traffic Pod Guest traffic

When adding a zone, you need to set up one or more physical networks. Each physical network can carry one or more types of traffic, with certain restrictions on how they may be combined. Add or remove one or more traffic types onto each physical network.

Network name	Isolation method	Traffic types	Tags
Physical Network 1	VLAN	MANAGEMENT (cloudbr0) + Add traffic	
Physical Network 2	VLAN	PUBLIC (cloudbr1) GUEST (cloudbr1)	

Add physical Network

Previous

Edit traffic type ⓘ

Please specify the traffic label you want associated with this traffic type.

* Traffic label:

Cancel OK

Next

10. Next, configure the *Public traffic* and add *public IPs* to be used by the *Virtual Routers* and *System VMs* on the public NIC. Complete the form as follows, click *Add*, and then click *Next*:

Gateway: *10.0.48.1*
 Netmask: *255.255.255.0*
 VLAN/VNI: *48*
 Start IP: *10.0.48.2*
 End IP: *10.0.48.254*

Add zone

✓

✓

✓

4

5

6

Zone typeCore zone typeZone detailsNetworkAdd resourcesLaunch

Physical NetworkPublic trafficPodGuest traffic

Public traffic is generated when Instances in the cloud access the internet. Publicly-accessible IPs must be allocated for this purpose. End Users can use the CloudStack UI to acquire these IPs to implement NAT between their guest Network and their public Network.

Provide at least one range of IP addresses for internet traffic.

Gateway	Netmask	VLAN/VNI	Start IP	End IP	
10.0.48.1	255.255.255.0	48	10.0.48.2	10.0.48.254	

1

Gateway

Netmask

VLAN/VNI

Start IP

End IP

2

Add

Previous

3

Next

Note

In this example, we're using VLAN 48 for the Public Network, but your setup might require a different VLAN or even no VLAN, depending on your network configuration. Adjust these values as needed for your specific environment.

11. Complete the form to create and setup the POD, then click [Next](#).

Note

A Pod represents a Rack in the Data Center. The network IPs used will be from the management network. Make sure to adjust these values to fit your specific network configuration.

Pod name: *POD1*
Rerved system gateway: *10.0.32.1*
Rerved system netmask: *255.255.240.0*
Start Rerved system IP: *10.0.34.1*
End Rerved system IP: *10.0.34.40*



Add zone ?

Zone type Core zone type Zone details **Network** Add resources Launch

Physical Network Public traffic Pod Guest traffic

Each zone must contain one or more pods. We will add the first pod now. A pod contains hosts and primary storage servers, which you will add in a later step. First, configure a range of reserved IP addresses for CloudStack's internal management traffic. The reserved IP range must be unique for each zone in the cloud.

* Pod name: ✓

* Reserved system gateway: ✓

* Reserved system netmask: ✓

* Start reserved system IP: ✓

* End reserved system IP: ✓

Previous **Next**

12. Next, configure the *Guest Traffic* VLANs, then click *Next*.

VLAN/VNI Range: *101 - 120*

Add zone ?

Zone type Core zone type Zone details **Network** Add resources Launch

Physical Network Public traffic Pod Guest traffic

Guest Network traffic is communication between end-user Instances. Specify a range of VLAN IDs or VXLAN Network identifiers (VNIs) to carry guest traffic for each physical Network.

VLAN/VNI range: ✓ - ✓

Previous **Next**

13. Next, complete the form with the *Cluster name*, then click *Next*.

Cluster Name: *cluster01*

Add zone

Zone type Core zone type Zone details Network **Add resources** Launch

Cluster IP address Primary storage Secondary storage

Each pod must contain one or more clusters. We will add the first cluster now. A cluster provides a way to group hosts. The hosts in a cluster all have identical hardware, run the same hypervisor, are on the same subnet, and access the same shared storage. Each cluster consists of one or more hosts and one or more primary storage servers.

* Cluster name: cluster01

Previous Next

14. Add the first host, then click *Next*. Additional hosts will be added once the Zone is ready.

Host Name: *host01.local*

Username: *root*

Password: *<the root password>*

Tags: Blank

Add zone

Zone type Core zone type Zone details Network **Add resources** Launch

Cluster IP address Primary storage Secondary storage

Each cluster must contain at least one host (computer) for guest instances to run on. We will add the first host now. For a host to function in CloudStack, you must install hypervisor software on the host, assign an IP address to the host, and ensure the host is connected to the CloudStack management server.

Give the host's DNS or IP address, the user name (usually root) and password, and any labels you use to categorize hosts.

* Host name: host01.local

* Username: root

Authentication Method: Password System SSH Key

* Password: *****

Tags:

Previous Next

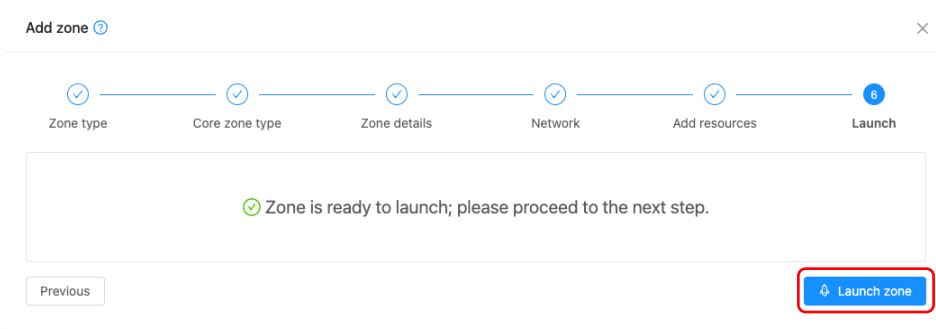
15. Complete the form as follows to add a NFS *Primary Storage*, then click *Next*.

Name: *Primary Storage*
 Scope: *Cluster*
 Protocol: *nfs*
 Server: *192.168.32.10*
 Path: */export/primary*
 NFS mount options: Blank
 Provider: *DefaultPrimary*
 Storage Tags: Blank

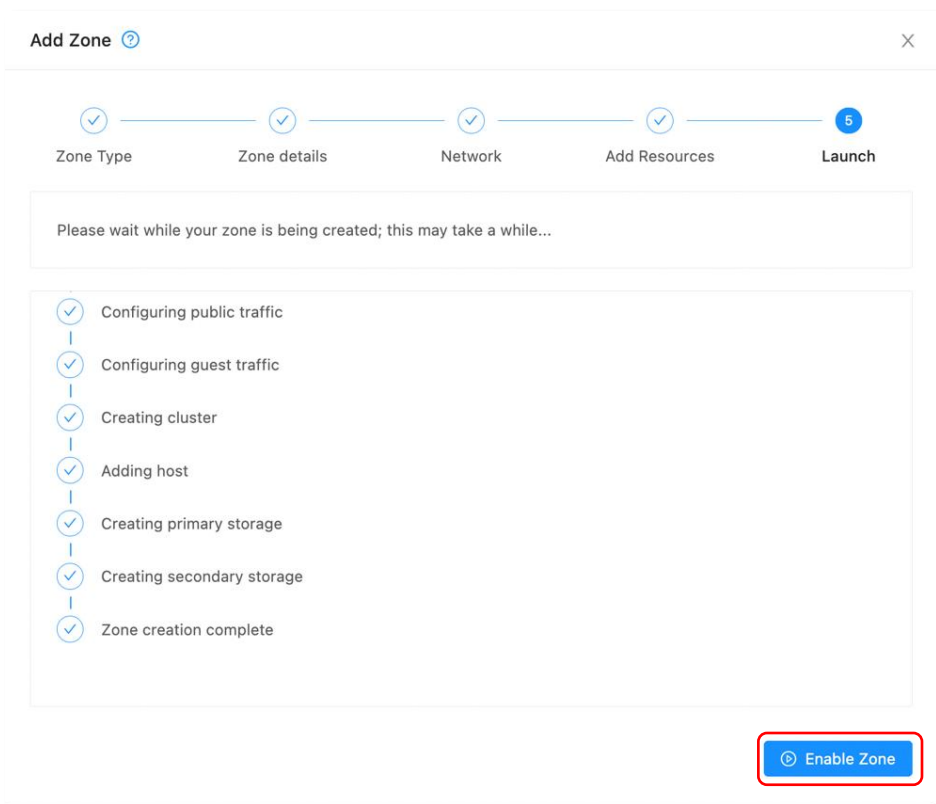
16. Complete the form as follows to add a NFS *Secondary Storage*, then click *Next*.

Provider: *NFS*
 Name: *Secondary Storage*
 Server: *192.168.32.10*
 Path: */export/secondary*

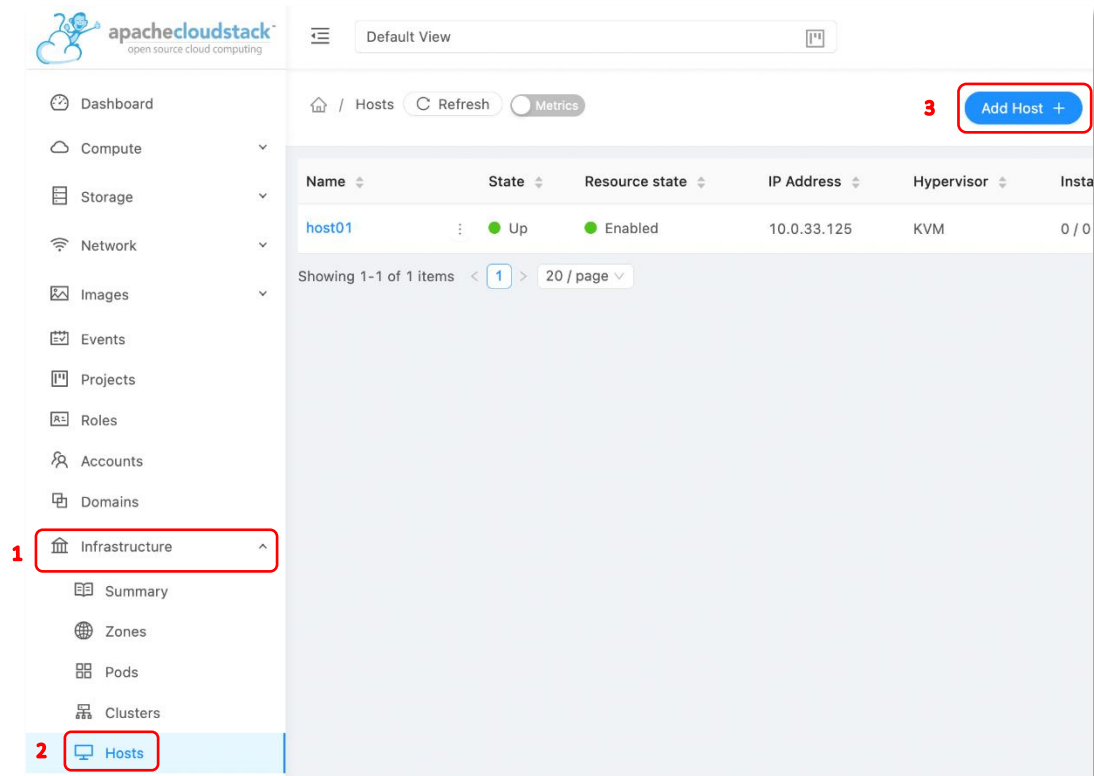
17. Click [Launch Zone](#) to proceed.



18. The Zone will be ready after all resources is configured. Click [Enable Zone](#) to finish the wizard.



19. Add the remaining hosts by clicking [Infrastructure](#) > [Hosts](#) in the left navigation pane, then click [Add Host](#).



20. Fill in the form as follow and click *OK*.

Zone Name: *Poc Zone*
Pod name: *POD1*
Cluster name: *cluster01*
Host Name: *host02.local*
Username: *root*
Password: *<The root password>*
Host Tags: Blank

The screenshot shows the 'Add host' dialog box with the following fields and values:

- Zone name: Poc Zone
- Pod name: POD1
- Cluster name: cluster01
- Host name: the host URL
- Username: the username for the host; required to be passed for hyp...
- Authentication Method: Password (selected), System SSH Key
- Password: the password for the host; required to be passed for ...
- Host tags: list of tags to be added to the host
- Dedicated: ☐

Buttons: Cancel, OK

Repeat this step to add *host03*.

Domain Hierarchy, User Accounts and Resource Limits



Overview

Apache CloudStack implements domain hierarchies to logically isolate user accounts. This model can be used to define, for example, departments within the same organization if used to building on-premises clouds or, different customers when building public clouds. You could also have a specific domain for sales partners or customers for example.

A domain can contain multiple user accounts. In Apache CloudStack, a user account assumes a profile defined in roles. There is a set of predefined roles for the most common user profiles, these being the main ones:

Role	Description
Root Admin	Manages the entire platform, including physical and logical resources across all domains and accounts.
Domain Admin	Manages all logical resources within the domain and its sub-domains, including user accounts, sub-domains, and all related virtual computing resources.
User	Manages virtual computing resources associated with the user's own account.

Topics covered

- Managing Domains
- Setting Resource Limits
- Managing User Accounts

Accessing the Control Panel

21. To access the CloudStack UI, open the following URL in your web browser:

<http://10.0.32.10:8080/client>

22. Login using the following credentials, then click [Login](#).

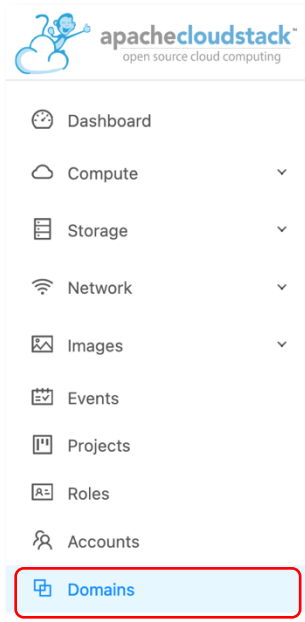
Username: *admin*
 Password: *password*
 Domain: Blank

Creating a Domain

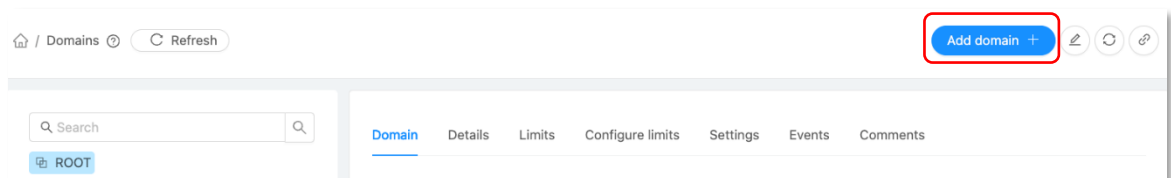
Note

In the domain and user accounts level, it is possible to distribute the compute resources to be used for the users, limiting their compute capacity. When a domain account is allocated at the domain level, the domain administrator user can distribute the resources for other regular user accounts and the adjacent sub-domains. It allows users to have granular control over allocated resources in many hierarchical levels.

1. In the left of navigation pane, click [Domains](#).

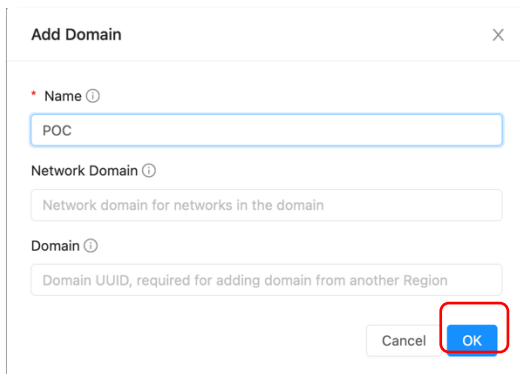


2. Next, click [Add Domain](#).



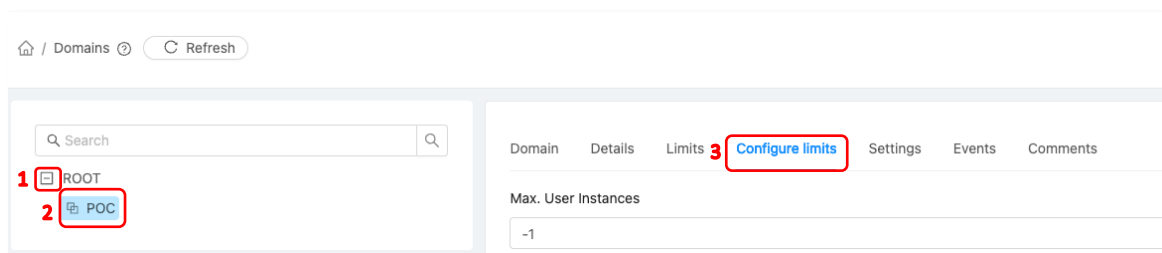
3. Complete the form as follows, then click [OK](#):

Name: *POC*
 Network Domain: Blank
 Domain: Blank



Limiting Resources at the Domain Level

4. Expand the domain tree, click on the *POC* Domain, then click [Configure Limits](#).



Note

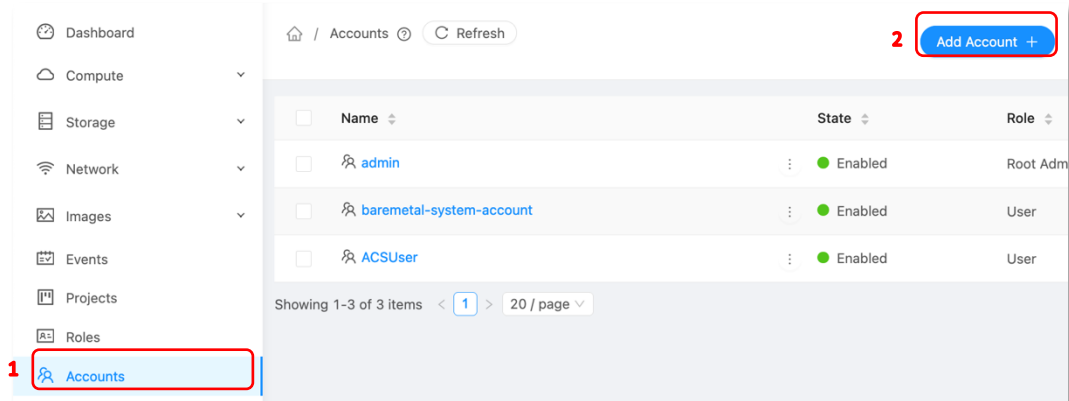
This screen displays the computing resource limits at the domain level. By default, the resources are set to *-1*, which indicates *unlimited resource*.

5. Set up the domain resources as follows, then click [Submit](#):

Max. User VMs: *10*
 Max. Public IPs: *5*
 Max. Volumes: *-1*
 Max. Snapshots: *-1*
 Max. Templates: *-1*
 Max. Networks: *5*
 Max. VPCs: *2*
 Max. CPU Cores: *-1*
 Max. Memory (MiB): *4096*
 Max. Primary Storage (GiB): *50*
 Max. Secondary Storage (GiB): *-1*

Adding a Domain Administrator Account

6. In the left navigation pane, click [Accounts](#), and then click [Add Account](#).



7. Fill in the form as follows and then click [OK](#).

Role: *Domain Admin (Domain Admin)*
 Username: *admin*
 Password: *password*
 Confirm Password: *password*
 Email: *admin@poc.zone*
 First Name: *Administrator*
 Last Name: *PoC Domain*
 Domain: *ROOT/POC*
 Account: *admin*
 Timezone: *<select your timezone>*
 Network Domain: Blank

Add Account

* Role: Domain Admin (DomainAdmin)

* Username: admin

* Password: * Confirm Password:

* Email: admin@poc.local

* First Name: Administrator * Last Name: PoC Domain

* Domain: ROOT/POC

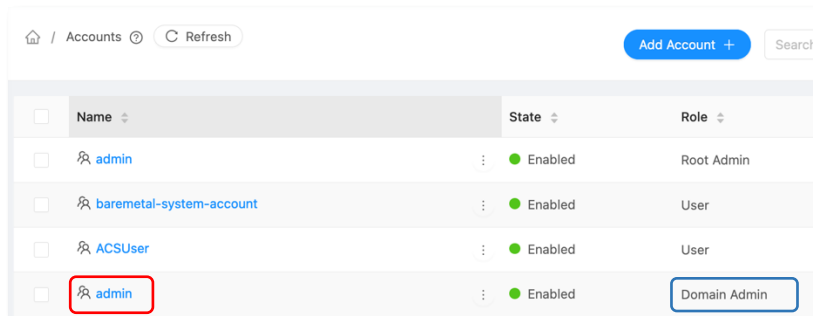
Account: admin

Timezone: GMT [Greenwich Mean Time]

Network Domain: Network domain for the account's networks

Cancel OK

8. Click the *admin* account with the *Domain Admin* role.



Accounts

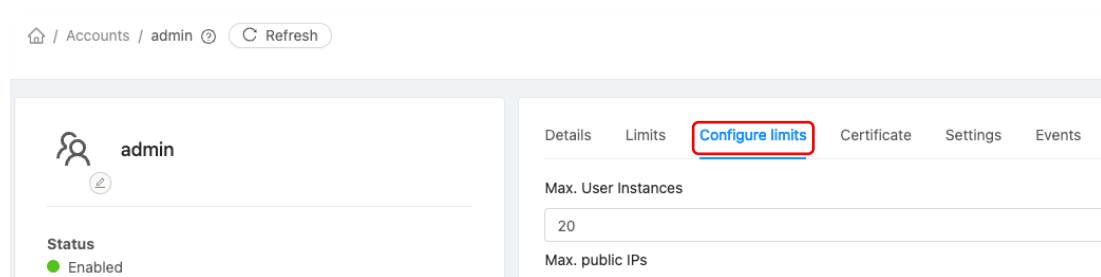
Refresh

Add Account +

Search

	Name	State	Role
☐	admin	Enabled	Root Admin
☐	baremetal-system-account	Enabled	User
☐	ACSUser	Enabled	User
☐	admin	Enabled	Domain Admin

9. Click the *Configure Limits* tab.



Accounts / admin

Refresh

admin

Status: Enabled

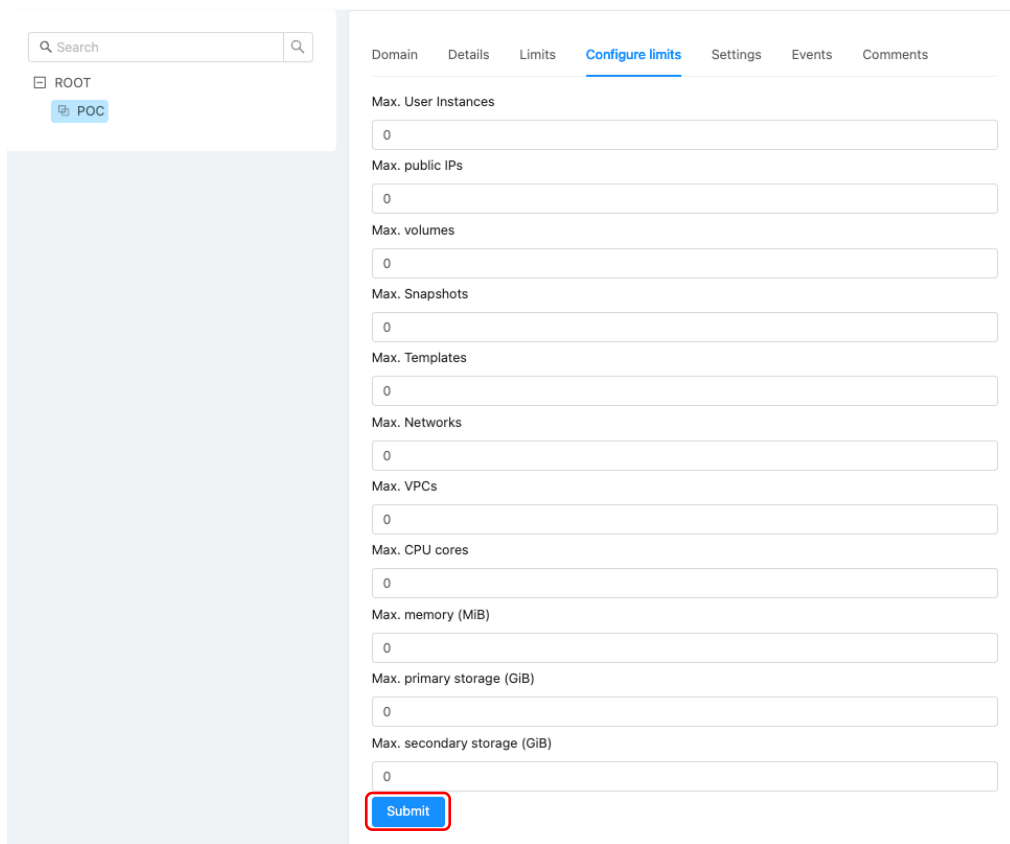
Details Limits **Configure limits** Certificate Settings Events

Max. User Instances: 20

Max. public IPs

10. The domain admin account will be used solely for administrative purposes, so no resources will be allocated to this account. Fill in the form as follows, then click [Submit](#).

Max. User VMs:	0
Max. Public IPs:	0
Max. Volumes:	0
Max. Snapshots:	0
Max. Templates:	0
Max. Networks:	0
Max. VPCs:	0
Max. CPU Cores:	0
Max. Memory (MiB):	0
Max. Primary Storage (GiB):	0
Max. Secondary Storage (GiB):	0



The screenshot shows the AWS Management Console interface for configuring resource limits. On the left, a domain hierarchy is shown with 'ROOT' and 'POC' (selected). The main panel has tabs for 'Domain', 'Details', 'Limits', 'Configure limits' (active), 'Settings', 'Events', and 'Comments'. Under the 'Configure limits' tab, a list of resource limits is displayed, each with a text input field containing the value '0':

- Max. User Instances
- Max. public IPs
- Max. volumes
- Max. Snapshots
- Max. Templates
- Max. Networks
- Max. VPCs
- Max. CPU cores
- Max. memory (MiB)
- Max. primary storage (GiB)
- Max. secondary storage (GiB)

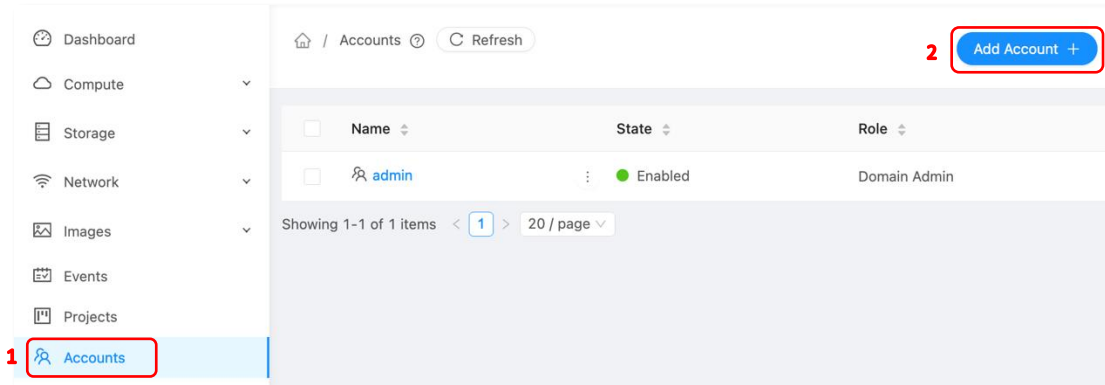
A red box highlights the 'Submit' button at the bottom of the form.

Adding a User Account and Setting Limits

11. Logout and log in using the following credentials:

Username: *admin*
 Password: *password*
 Domain: *poc*

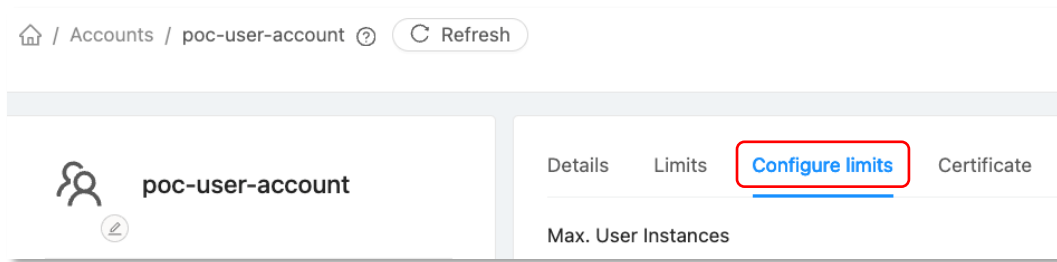
12. In the left navigation pane, click [Accounts](#), and then click [Add Account](#).



13. Complete the form as follows, then click [OK](#):

Role: *User*
 Username: *user-1*
 Password: *password*
 Confirm Password: *password*
 Email: *user-1@poc.zone*
 First Name: *User-1*
 Last Name: *PoC Domain*
 Domain: *ROOT/POC*
 Account: *poc-user-account*
 Timezone: *<select yours>*
 Network Domain: *Blank*

14. Select *poc-user-account*, and then click *Configure Limits*.



Home / Accounts / poc-user-account ? Refresh

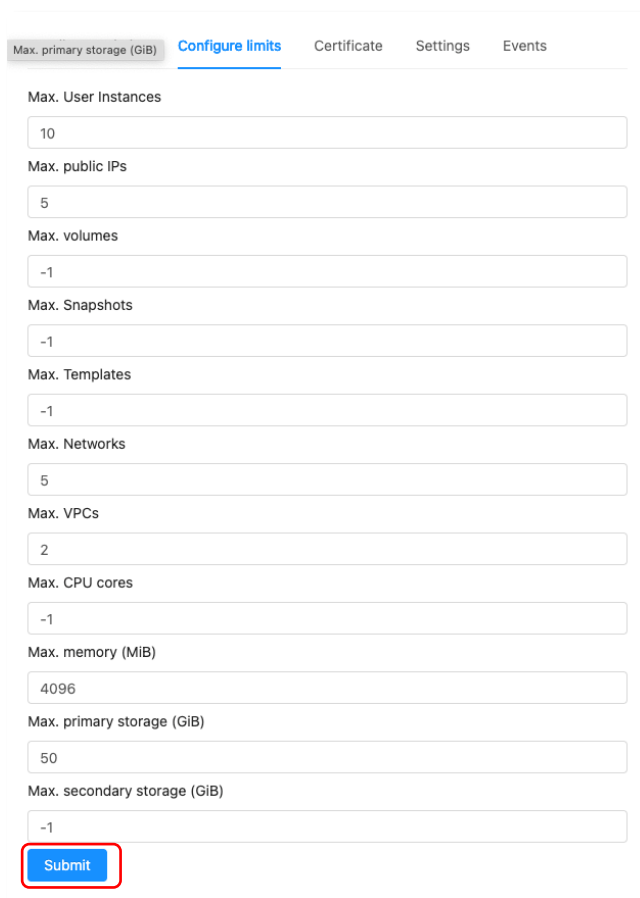
 **poc-user-account** 

Details Limits **Configure limits** Certificate

Max. User Instances

15. Fill in the form as follows and then click *Submit*:

Max. User VMs:	10
Max. Public IPs:	5
Max. Volumes:	-1
Max. Snapshots:	-1
Max. Templates:	-1
Max. Networks:	5
Max. VPCs:	2
Max. CPU Cores:	-1
Max. Memory (MiB):	4096
Max. Primary Storage (GiB):	50
Max. Secondary Storage (--GiB):	-1



Max. primary storage (GiB) **Configure limits** Certificate Settings Events

Max. User Instances

10

Max. public IPs

5

Max. volumes

-1

Max. Snapshots

-1

Max. Templates

-1

Max. Networks

5

Max. VPCs

2

Max. CPU cores

-1

Max. memory (MiB)

4096

Max. primary storage (GiB)

50

Max. secondary storage (GiB)

-1

Submit

Managing Guest Instances



Overview

This instruction provides you with a basic overview of launching, resizing, and managing an Apache CloudStack Instance.

Apache CloudStack offers a user-friendly interface that simplifies obtaining and configuring capacity. It gives users full control over their computing resources and operates within a reliable, proven environment.

CloudStack significantly reduces the time needed to provision and boot new virtual machine Instances to just minutes, enabling you to quickly scale capacity up or down as your computing requirements change.

Topics Covered

- Download Template
- Create an Isolated Network
- Modify egress firewall rules
- Launch an Instance
- Using Userdata
- Manage Firewall
- Scale up/down Instance
- Instance console access
- Destroy Instance
- Recover Instance
- Expunge Instance

Accessing the Control Panel

1. To access the CloudStack UI, open the following URL in your web browser:

<http://10.0.32.10:8080/client>

2. Login using the following credentials, then click [Login](#).

Username: *user-1*
Password: *password*
Domain: *poc*

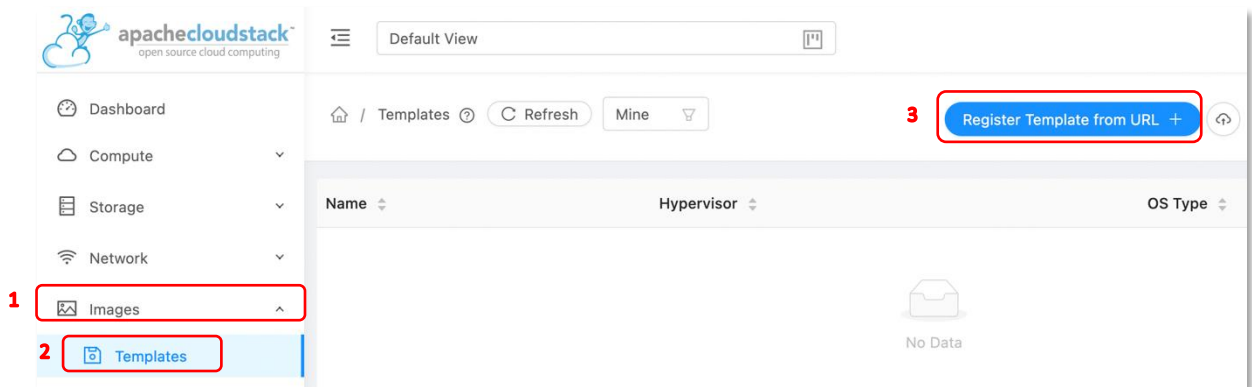


Registering a new Template

Note

An Apache CloudStack template provides a root disk with a pre-installed operating system for launching an Instance, which is a virtual machine in the cloud. The template may also include a pre-installed and configured application server.

3. In the left navigation pane, click *Images* > *Templates*, and then click *Register Template from URL*.

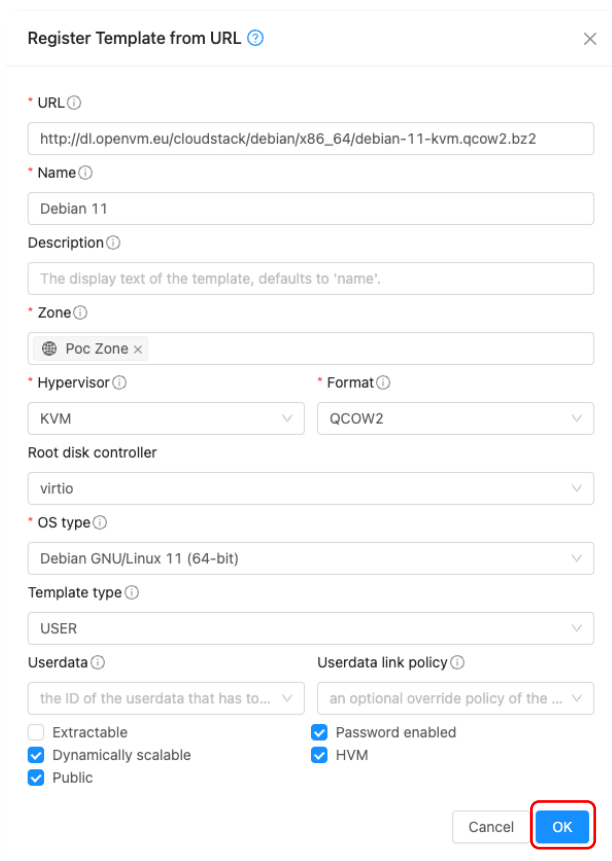


Note

You will use a pre-configured template that includes cloud-init, which is set up to run commands during Instance startup.

4. Complete the form as follows, then click **OK**:

URL: **http://dl.openvm.eu/cloudstack/debian/x86_64/debian-11-kvm.qcow2.bz2**
 Name: **Debian 11**
 Description: **Debian GNU/Linux 11 (64-bit)**
 Zone: **poc-zone**
 Hypervisor: **KVM**
 Format: **QCOW2**
 Root disk controller: **virtio**
 OS Type: **Debian GNU/Linux 11 (64-bit)**
 Template Type: **USER**
 Userdata: **None**
 Userdata link policy: **None**
 Extractable: **No**
 Dynamically Scalable: **Yes**
 Public: **Yes**
 Password Enabled: **Yes**
 HVM: **Yes**

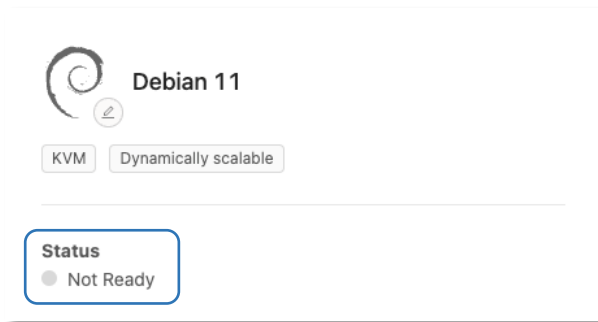


The screenshot shows the 'Register Template from URL' dialog box. The fields are as follows:

- URL:** http://dl.openvm.eu/cloudstack/debian/x86_64/debian-11-kvm.qcow2.bz2
- Name:** Debian 11
- Description:** The display text of the template, defaults to 'name'.
- Zone:** Poc Zone
- Hypervisor:** KVM
- Format:** QCOW2
- Root disk controller:** virtio
- OS type:** Debian GNU/Linux 11 (64-bit)
- Template type:** USER
- Userdata:** the ID of the userdata that has to...
- Userdata link policy:** an optional override policy of the ...
- Extractable:** ☐
- Dynamically scalable:** ☒
- Public:** ☒
- Password enabled:** ☒
- HVM:** ☒

The 'OK' button is highlighted with a red rectangle.

- From the previously registered templates, select [Debian 11](#) to check its status:



- Refresh the page by clicking [refresh](#) until the status shows as ready:



Creating a Guest Network and Managing Egress Rules

Note

At the domain and user account levels, you can allocate compute resources and set limits on compute capacity. When resources are allocated at the domain level, the domain administrator can distribute them to other regular user accounts and adjacent sub-domains. This setup allows for granular control over resource allocation across multiple hierarchical levels.

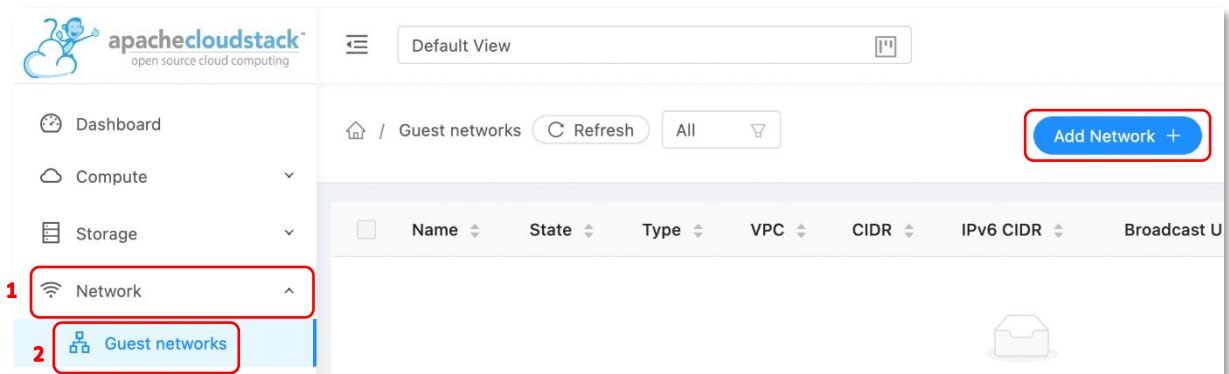
Note

An Apache CloudStack guest network provides a VLAN-isolated layer that connects the Instance network to the Virtual Router gateway.

Note

The Virtual Router handles network communication between Instances and the public network. It also manages firewall rules, userdata, metadata, DHCP for Instances, load balancing, Remote VPN, and TCP/UDP port forwarding.

7. In the left navigation pane, click [Networks](#) > [Guest networks](#), and then click [Add Network](#).



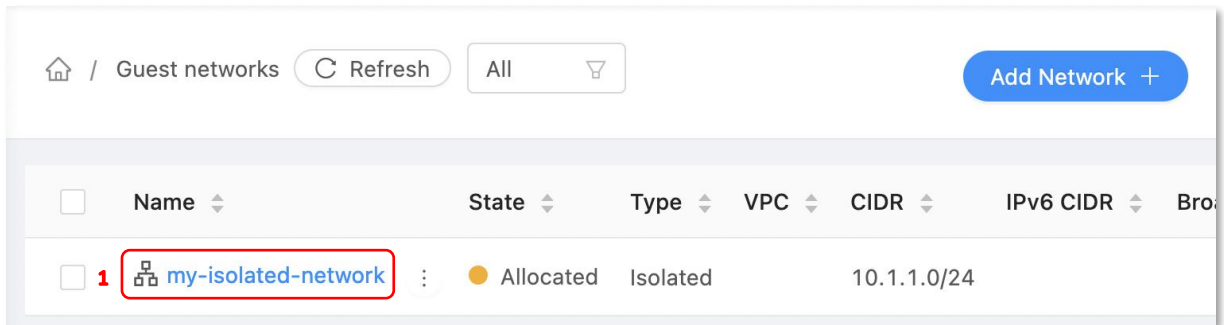
8. Select [Isolated](#), and fill in the form as follows and then click [OK](#):

Name: my-isolated-network
 Description: My Isolated Network
 Zone: Poc Zone
 Network Offering: Offering for Isolated networks with Source Nat service enabled
 External Id: Blank
 Gateway: Blank
 Netmask: Blank
 DNS 1: Blank
 DNS 2: Blank
 Ipv4 address for the VR in this network: Blank
 Network Domain: Blank

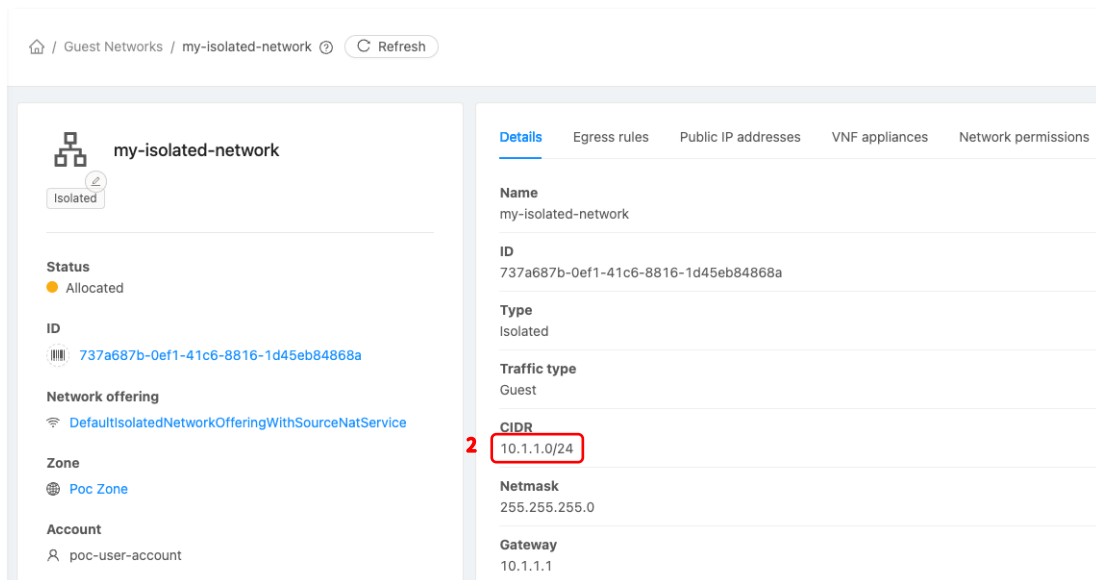
Note

During the creation of an *Isolated Network*, the parameters *Gateway*, *Netmask*, *DNS 1*, *DNS 2*, and *Ipv4 address for the VR in this network* can be customized. However, if left blank, these parameters will inherit the default values from the Zone configuration.

9. Click on *my-isolated-network* and then copy the CIDR address.



	Name	State	Type	VPC	CIDR	IPv6 CIDR	Bro
☐ 1	my-isolated-network	Allocated	Isolated		10.1.1.0/24		



my-isolated-network

Isolated

Status
Allocated

ID
737a687b-0ef1-41c6-8816-1d45eb84868a

Network offering
DefaultIsolatedNetworkOfferingWithSourceNatService

Zone
Poc Zone

Account
poc-user-account

Details | Egress rules | Public IP addresses | VNF appliances | Network permissions

Name
my-isolated-network

ID
737a687b-0ef1-41c6-8816-1d45eb84868a

Type
Isolated

Traffic type
Guest

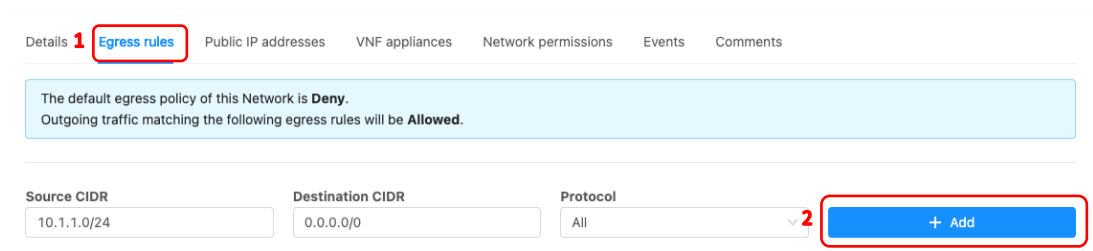
CIDR
10.1.1.0/24

Netmask
255.255.255.0

Gateway
10.1.1.1

10. Click *Egress Rules* and add a rule to allow Instances to access the internet as follows. Then click *Add*.

Source CIDR: 10.1.1.0/24
 Destination CIDR: 0.0.0.0/0
 Protocol: All



Details 1 **Egress rules** | Public IP addresses | VNF appliances | Network permissions | Events | Comments

The default egress policy of this Network is **Deny**.
Outgoing traffic matching the following egress rules will be **Allowed**.

Source CIDR 10.1.1.0/24	Destination CIDR 0.0.0.0/0	Protocol All	+ Add
----------------------------	-------------------------------	-----------------	--------------

11. Below, you can see the added egress rule.

The default egress policy of this Network is **Deny**.
Outgoing traffic matching the following egress rules will be **Allowed**.

Source CIDR	Destination CIDR	Protocol	Start port	End port	
		TCP			+ Add

☐	Source CIDR	Destination CIDR	Protocol	ICMP type / Start port	ICMP code / End port	Actions
☐	10.1.1.0/24	0.0.0.0/0	All	All	All	

Total 1 items < 1 > 10 / page

Registering a UserData

Note

When creating an Instance, you can use Managed UserData to customize its configuration during the boot process. Managed UserData lets you define settings, scripts, or commands that are automatically applied when the Instance starts. This can include tasks such as installing software, configuring services, or setting environment variables.

12. In the left navigation pane, click [Compute](#) > [User Data](#), then click [Register a userdata](#).

1 [Compute](#)

2 [User Data](#)

3 [Register a userdata +](#)

Name	ID
No Data	

Showing 0-0 of 0 items < 1 > 20 / page

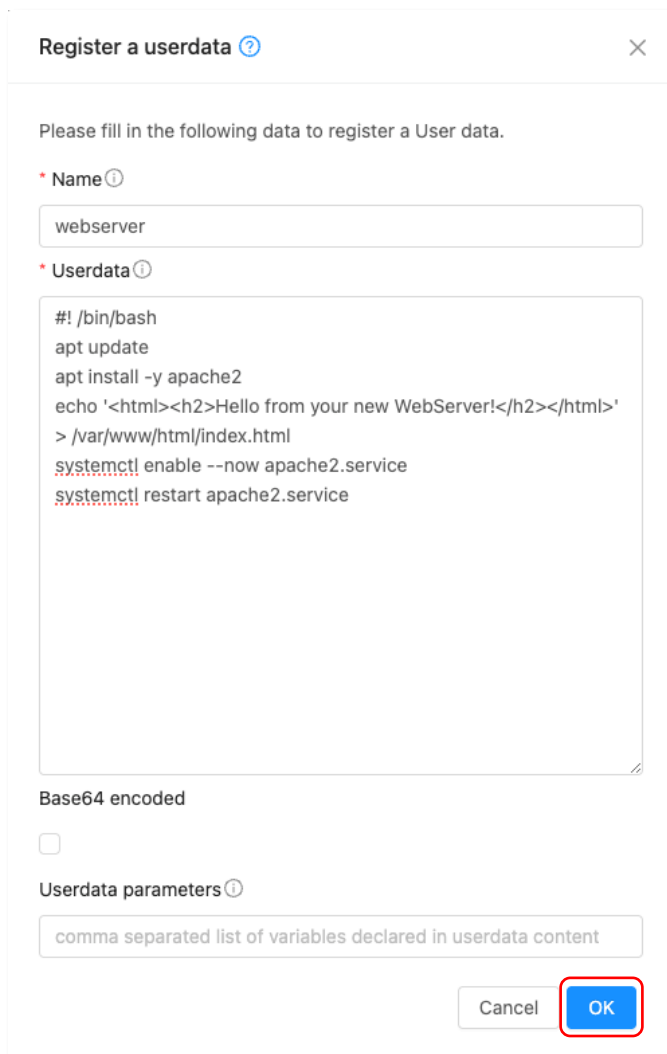
13. Complete the form as follows, then click [OK](#).

Name: *webserver*

Userdata: *#!/bin/bash*
apt update
apt install -y apache2
echo '<html><h2>Hello from your new WebServer!</h2></html>' >
/var/www/html/index.html
systemctl enable --now apache2.service
systemctl restart apache2.service

Base64 encoded: No

Userdata parameters: Blank



Register a userdata ⓘ

Please fill in the following data to register a User data.

* Name ⓘ

webserver

* Userdata ⓘ

```
#!/bin/bash
apt update
apt install -y apache2
echo '<html><h2>Hello from your new WebServer!</h2></html>'
> /var/www/html/index.html
systemctl enable --now apache2.service
systemctl restart apache2.service
```

Base64 encoded

☐

Userdata parameters ⓘ

comma separated list of variables declared in userdata content

Cancel OK

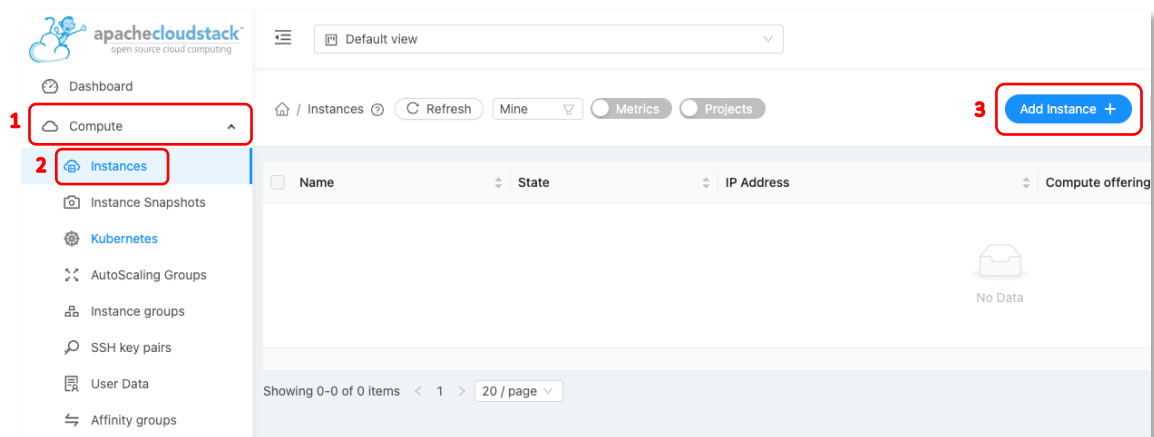
Creating an Instance running a Web Server

Note

When creating an Instance, you can use Managed UserData to customize its configuration during the boot process. Managed UserData allows you to define settings, scripts, or commands that are automatically applied when the Instance starts. This can include tasks like installing software, configuring services, or setting environment variables.

The cloud-init tool, which is available inside the Debian 11 Template we use, plays a crucial role in this process. Cloud-init fetches the UserData provided during Instance creation and processes it within the guest OS (the virtual machine). This UserData is consumed by cloud-init during the boot sequence, ensuring that the specified configurations and commands are executed automatically, simplifying the initialization and customization of Instances without manual intervention.

14. In the left navigation pane, click [Compute](#) > [Instances](#), then click [Add Instance](#).

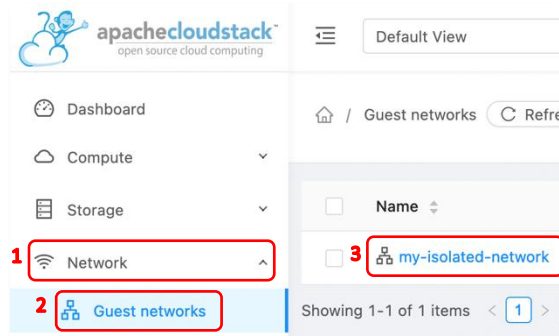


15. Complete the form as follows, then click [Launch Instance](#):

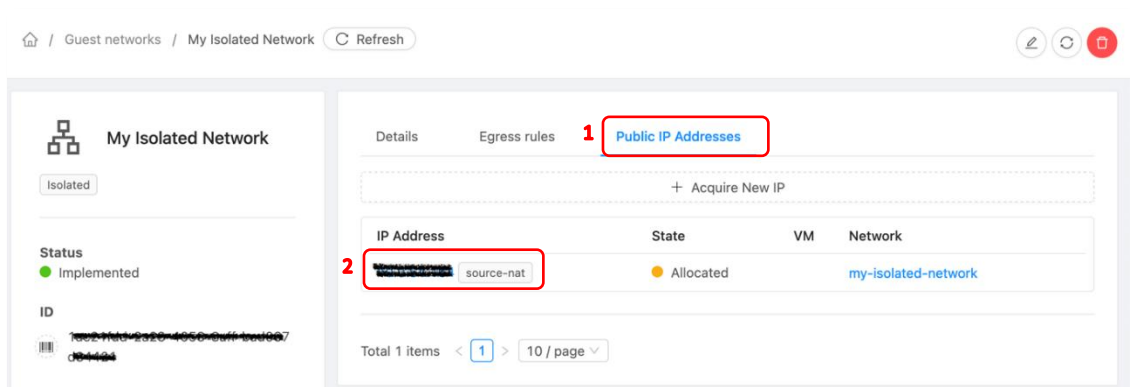
Zone:	poc-zone
Template:	Debian 11
Compute Offering:	Small Instance
Advanced Mode:	Yes
- Stored UserData:	webserver
Name:	WebServer01
Group:	None
Keyboard Language:	None
Start Instance:	Yes

Configuring Firewall and Port Forwarding for Public Web Server Access

16. In the left navigation pane, click *Network* > *Guest networks*, then click *my-isolated-network*.

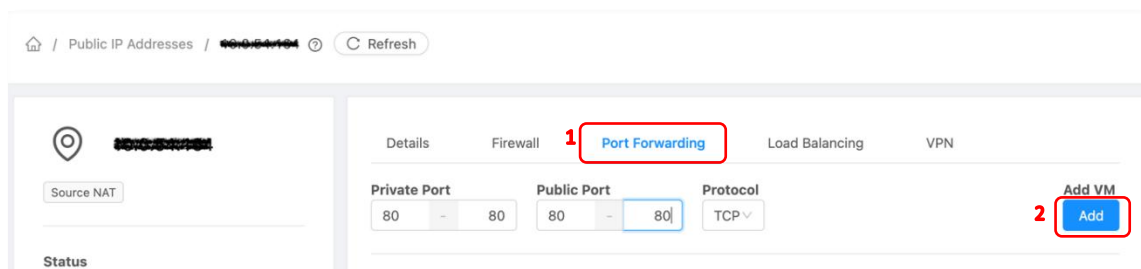


17. Select *Public IP Address*, then click on *source nat* IP Address.



18. Click the *Port Forwarding* tab and add a rule to redirect the HTTP port as follows, then click *Add*.

Private Port: Start: 80 – End: 80
Public Port: Start: 80 – End: 80



19. It will display a list of of Instances available on *my-isolated-network* network. Select *WebServer01* and click *OK*.

The 'Add VM' dialog box displays a table of instances. The instance 'WebServer01' is selected. The 'OK' button is highlighted with a red box and a '2'.

Name	State	Display Name	IP Address	Account	Zone	Select
WebServer01 1[redacted] (Primary)	Running	WebServer01		poc-user-account-1	PoC-Lab	1 [icon]

Total 1 items < 1 > 10 / page

Cancel 2 OK

20. Click the *Firewall* tab, complete the form as follows, and then click *Add*.

Source CIDR: 0.0.0.0/0
 Protocol: TCP
 Start Port: 80
 End Port: 80

The Firewall configuration form shows the 'Firewall' tab selected. The 'Add' button is highlighted with a red box and a '2'.

Details 1 Firewall Port Forwarding Load Balancing VPN

Source CIDR: 0.0.0.0/0 Protocol: TCP Start Port: 80 End Port: 80 2 Add

21. The added rule will be displayed in the firewall list.

Source CIDR	Protocol	Start Port/ICMP Type	End Port/ICMP Code	State	Action
0.0.0.0/0	TCP	80	80	Active	[icon] [icon]

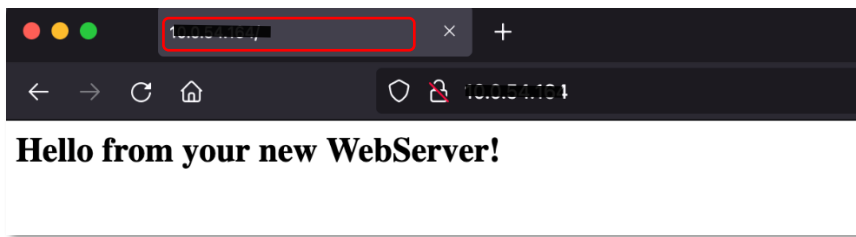
22. In the *Details* tab, copy the *IP address*.

The 'Details' tab is selected. The 'IP Address' field is highlighted with a red box and a '2'.

1 Details Firewall Port Forwarding Load Balancing VPN

2 IP Address [redacted]

23. Open a new browser tab, paste the copied address, and press enter:



Note

The Web Server is now running.

Scaling Up/Down Guest Instances

Note

Scaling out/in involves adjusting the number of Instances to handle changes in load or demand. Scaling out adds more Instances, while scaling in removes excess Instances. This approach ensures that your application can manage varying workloads efficiently.

In contrast, scaling up/down changes the allocated resources of existing Instances, such as increasing memory, CPU or disk. This method helps in handling increased demand without modifying the number of Instances.

Note

Compute offering in Apache CloudStack define the specifications for virtual machine Instances, such as CPU, memory, network rate, and disk size. These offerings are critical for ensuring that Instances have the necessary resources to meet performance and capacity requirements.

24. Logout and then login with the following credentials:

Username: *admin*
 Password: *password*
 Domain: *poc*

25. In the left navigation pane, click *Service Offering* > *Compute Offering*, then click *Add Compute Offering*.

Name	Description	State	CPU cores	CPU (in MHz)	Memory
Small Instance	Small Instance	Active	1	500	512
Medium Instance	Medium Instance	Active	1	1000	1024
MinimumCKS	MinimumCKS	Active	2	1500	2048
Large Instance	Large Instance	Active	4	2000	4096

26. Complete the form as follows, then click **OK**:

Name:	<i>Poc Instance</i>
Description:	<i>Personal PoC Instance</i>
Compute Offering Type:	<i>Fixed Offering</i>
CPU Cores:	<i>2</i>
CPU (in Mhz):	<i>1200</i>
Memory (in MB):	<i>768</i>
Host tags:	Blank
Network Rate (Mb/s):	Blank
Offer HA:	<i>Yes</i>
Dynamic scaling enable:	<i>Yes</i>
CPU Cap:	No
Volatile:	No
Deployment planner:	None
GPU:	None
Public:	No
Domain:	<i>ROOT/POC</i>
Zone:	<i>All Zones</i>
Compute only disk offering:	No
Disk Offering:	<i>Medium Disk, 20 GB</i>
Disk offering strictness:	No

The screenshot shows the 'Add compute offering' form in the AWS Management Console. The form is titled 'Add compute offering' and has a close button (X) in the top right corner. The form contains the following fields and values:

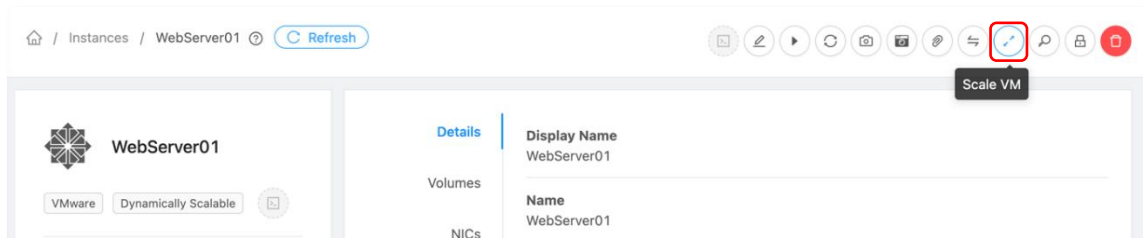
- Name:** Poc Instance
- Description:** Personal PoC Instance
- Compute offering type:** Fixed offering (selected), Custom constrained, Custom unconstrained
- CPU cores:** 2
- CPU (in Mhz):** 1200
- Memory (in MB):** 768
- Host tags:** the host tag for this service offering.
- Network rate (Mb/s):** data transfer rate in megabits per second allow...
- Offer HA:** ☒
- Dynamic scaling enabled:** ☒
- CPU cap:** ☐
- Volatile:** ☐
- Deployment planner:** (empty dropdown)
- GPU:** None
- Public:** ☐
- Domain:** ROOT/POC x
- Zone:** All zones x
- Compute only disk offering:** ☐
- Disk offerings:** Medium Disk, 20 GB
- Disk offering strictness:** ☐

At the bottom right of the form, there are two buttons: 'Cancel' and 'OK'. The 'OK' button is highlighted with a red box.

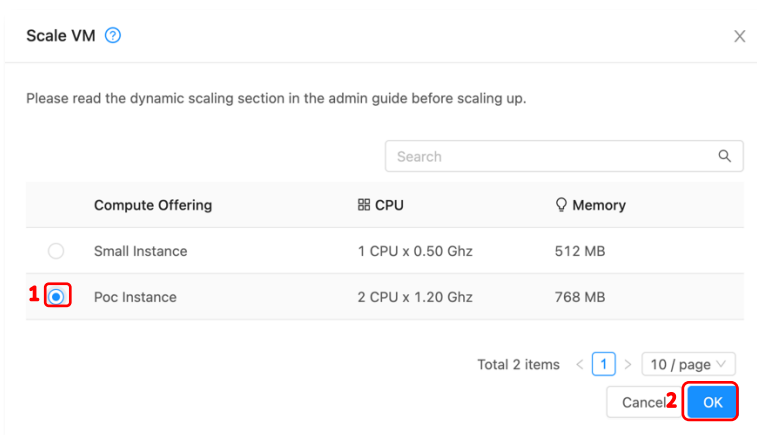
27. Logout the current session and login using the following credentials:

Username: *user-1*
 Password: *password*
 Domain: *poc*

28. Stop the Instance, then click *Scale Instance*.



29. Select the *Poc Instance* Compute Offering, then click *OK*.



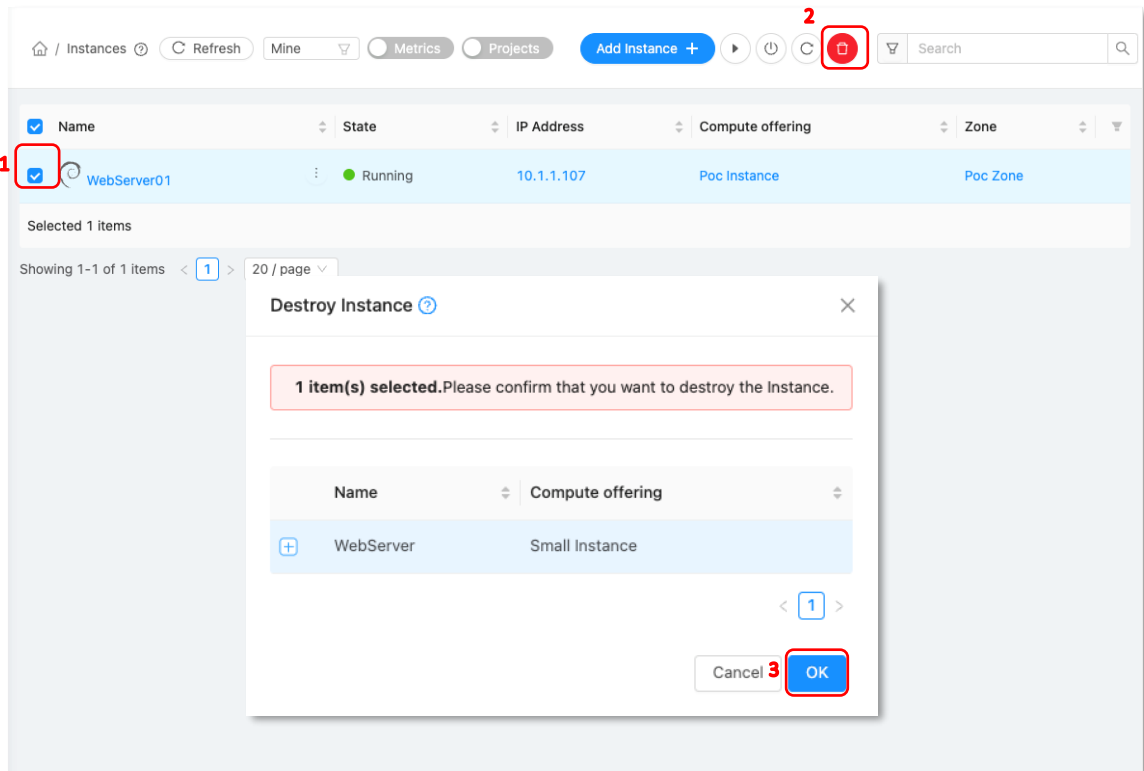
30. The Instance will be scaled with the values inherited from the Compute Offering *Poc Instance*.

Removing and Recovering Instances

Note

When an Apache CloudStack Instance is no longer needed, it can be destroyed by the user. By default, a normal user account can only destroy the Instance but cannot permanently remove it from the cloud infrastructure. The ability for users to expunge Instances is controlled by the Global Setting variable *allow.user.expunge.recover.vm*. If this variable is set to true, users can expunge Instances, which means they can permanently remove them from the cloud infrastructure. If no action is taken within the period defined by the Global Setting variable *event.purge.interval*, the event purge thread will permanently remove the resource from the infrastructure. Until this purge occurs, Domain and Account Limits continue to be enforced.

31. In the left navigation pane, click *Compute* > *Instances*, select *WebServer01*, click *Destroy Instance*, and then click *OK*.



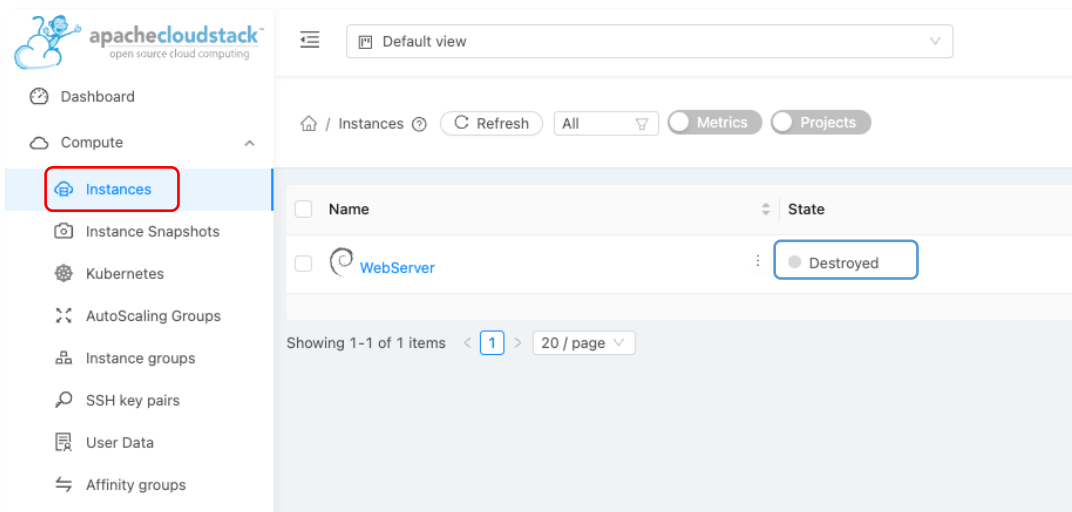
Note

When an Instance is destroyed by a regular user account, it may appear as permanently removed from the user's view. However, if the Global Setting *allow.user.view.destroyed.vm* variable is set to true, users can view and recover such destroyed Instances. By default, this setting is false, meaning users cannot see or recover destroyed Instances.

32. Logout the current session and login using the following credentials:

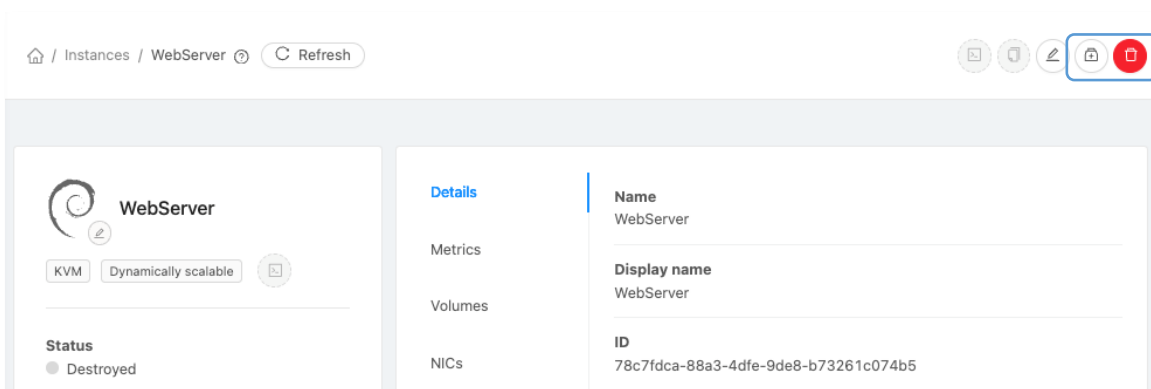
Username: *admin*
 Password: *password*
 Domain: *poc*

33. In the left navigation pane, click [Compute > Instances](#).

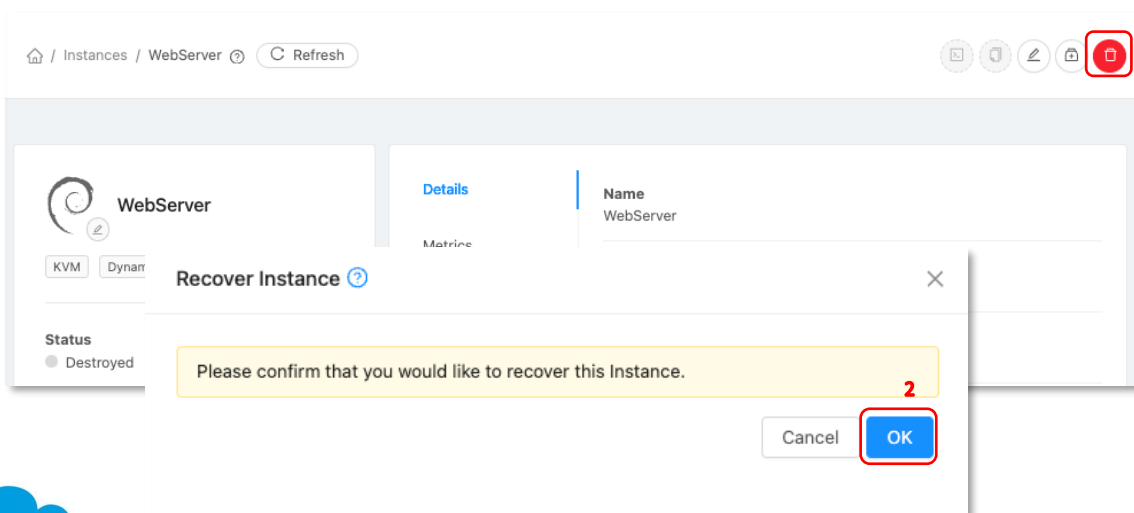


Note

A domain admin account has the capability to manage destroyed Instances, which includes the ability to both recover and permanently purge these Instances. This functionality allows domain admins to handle Instances that regular users have destroyed but not permanently removed. Here's what a domain admin can do:



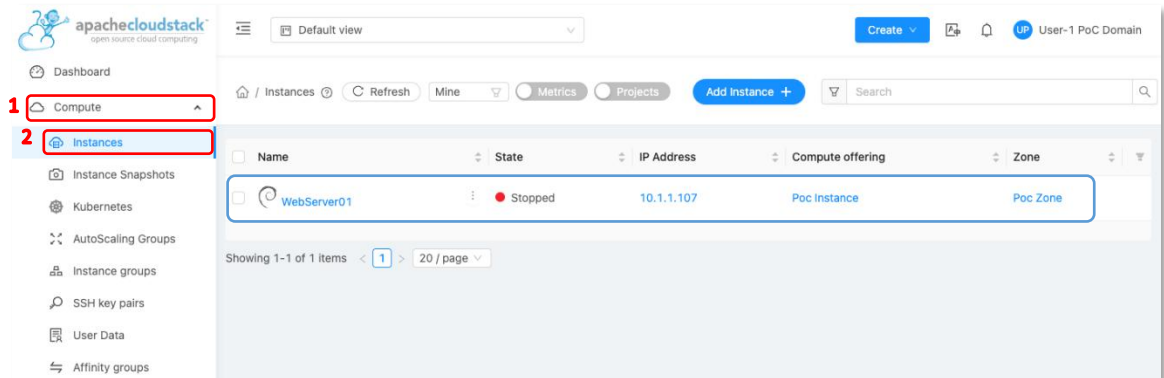
34. To recover an Instance that has been removed by a user, click [Recover Instance](#). Once this action is completed, the Instance will be restored and available again for use.



35. Logout, and then login with the following credentials:

Username: *user-1*
Password: *password*
Domain: *poc*

36. In the left navigation pane, click *Compute* > *Instances* to view the recovered Instance.



Virtual Private Cloud



Overview

Virtual Private Cloud (VPC) enables you to create an architecture that mimics a traditional physical network. Key features of VPC include:

Network Tier isolation

Separates network traffic into distinct layers or network “tiers” providing multiple isolated network segments within a VPC. This structure enhances security by allowing each tier to serve a specific function (such as public-facing services, backend operations, or database access) and simplifies network management.

Access Control List (ACL)

Provides granular control over network traffic by defining rules that permit or deny data flows.

Site-to-site IPsec VPN

Establishes secure connections between your on-premises infrastructure and your VPC using IPsec protocols.

Client VPN

Enables remote users to securely connect to your network using a VPN client. This feature is available for both VPCs and Isolated Network setups, offering secure access for remote connections regardless of the network structure.

Internal and External Load Balancer

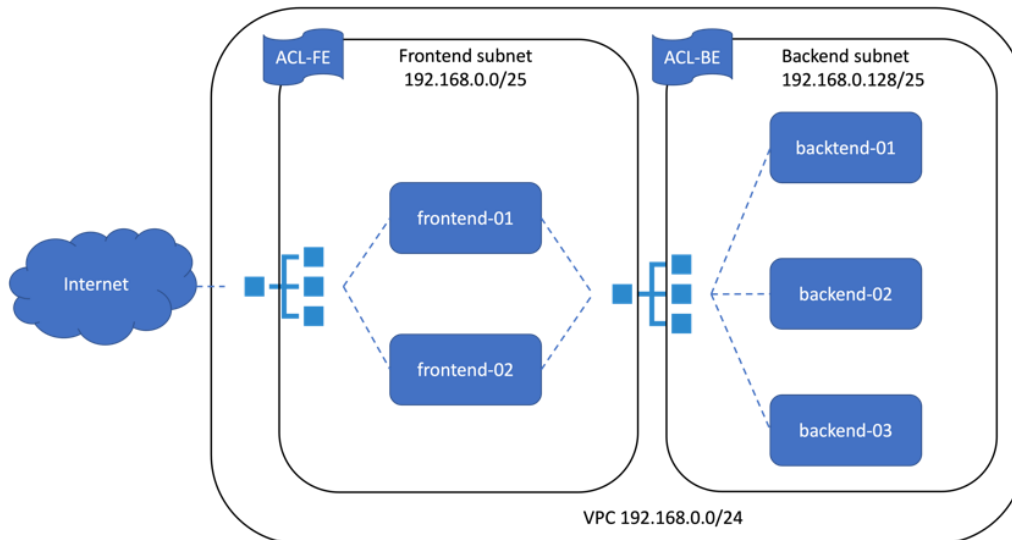
Distributes incoming traffic across multiple Instances to ensure high availability and reliability of applications, both within the VPC and for external access.

Topics covered

- How to create VPC
- How to create ACLs List
- How to create VPC subnets
- How to create internal and external Load Balancer

VPC Architecture Overview





In this step, you will design and implement a simple architecture to showcase the capabilities of Virtual Private Cloud (VPC) components for delivering services. Here's how the architecture will be structured:

- **Define VPC CIDR:** Allocate a CIDR block for your VPC to segment network traffic. This CIDR block will be further divided into subnets to organize and manage traffic flow, by creating different Networks inside VPC.
- **Subnetting**
 - **Frontend Subnet:** (192.168.0.0/25) This subnet will manage incoming traffic from the internet, distributing it to frontend instances through load balancing to ensure high availability and efficient content delivery.
 - **Backend Subnet:** (192.168.0.128/25): This subnet will handle processing and serving content, accessible only via the frontend subnet. Internal load balancing will manage traffic between backend instances for scalable processing.
- **Instances**
 - **Frontend Instances:** Deploy 2 Instances in the frontend subnet. These will handle external user requests and distribute traffic.
 - **Backend Instances:** Deploy 3 Instances in the backend subnet. These will process requests and provide content.
- **Load Balancing:**
 - **External Load Balancer:** Set up a load balancer in the frontend subnet to distribute incoming traffic across the frontend Instances using a round-robin algorithm.

- **Internal Load Balancer:** Configure an internal load balancer in the backend subnet to distribute traffic among backend Instances. This load balancer is only accessible from the frontend subnet.
- **Request Handling:**
 - The external load balancer will direct incoming user requests to the frontend Instances.
 - Each frontend Instance will use the internal load balancer to distribute traffic to the backend Instances.
 - The round-robin algorithm will ensure that each request is distributed evenly across the Instances, allowing for balanced load handling and high availability.

Creating a VPC

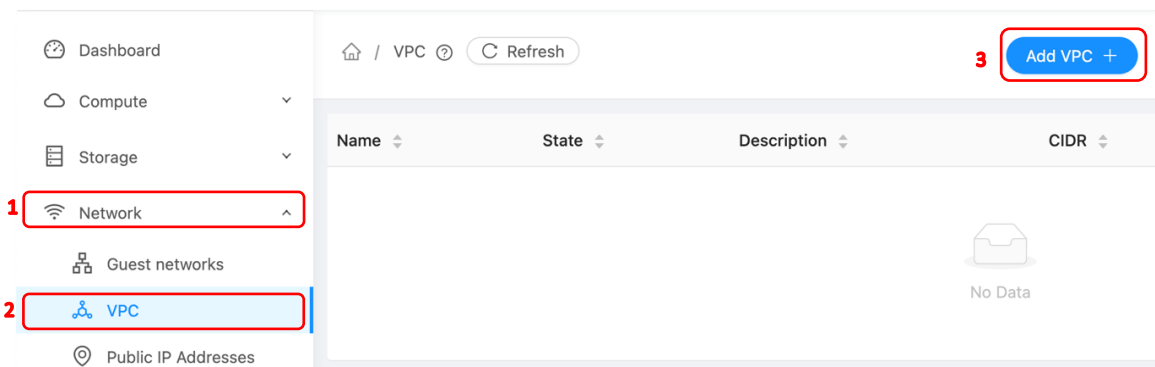
Login with the following credentials:

Username: *user-1*

Password: *password*

Domain: *poc*

1. In the left navigation pane, click *Network* > *VPC*, then click *Add VPC*.



2. Complete the form as follows, then click *OK*.

Name:	<i>My VPC</i>
Description:	Blank
Zone:	<i>poc-zone</i>
CIDR:	<i>192.168.0.0/24</i>
Network Domain:	Blank
VPC Offering:	<i>Default VPC Offering</i>
DNS 1:	Blank
DNS 2:	Blank
IPv4 address for the VR in this Network:	Blank
Start:	<i>yes</i>

Add VPC ⓘ

* Name ⓘ
My VPC

Description ⓘ
My VPC

* Zone ⓘ
Poc Zone

* CIDR ⓘ
192.168.0.0/24

Network domain ⓘ
VPC network domain. All networks inside the VPC will belong to this domain

* VPC offering ⓘ
Default VPC offering

DNS 1 ⓘ
the first IPv4 DNS for the VPC

DNS 2 ⓘ
the second IPv4 DNS for the VPC

IPv4 address for the VR in this Network. ⓘ
IPv4 address to be assigned to the public interface of the network router. This address ...

Start ⓘ
☒

Cancel OK

Creating ACL Lists

In Apache CloudStack, a Network ACL (Access Control List) is a set of ordered rules that control the flow of traffic within a Virtual Private Cloud (VPC). These rules are evaluated sequentially, starting from the lowest number, to determine whether traffic is permitted or denied. ACLs are associated with VPC Network tiers and can be applied to multiple network tiers, but each tier can only have one ACL assigned at a time. The ACL rules specify the types of traffic, sources, destinations, and protocols allowed or denied, providing a secure, organized structure for traffic management between different network tiers within the VPC.

Note

Before setting up the VPC networks, we will first create the necessary ACLs to control access for each VPC network tier.

CloudStack offers two default ACLs:

- **default_allow**: Allows all inbound and outbound traffic.
- **default_deny**: Denies both inbound and outbound traffic.

If you don't have a dedicated/custom-made ACL in advance, you must select one of these defaults two ACLs while creating VPC Networks. The default Network ACLs cannot be modified or deleted. You can also do it the other way around – first create VPC networks, use one of the default ACLs, and later create your own ACLs and apply them for each VPC Network Tier (replacing the currently applied default ACL for each VPC network tier), to fine-tune traffic control according to your requirements. However, as stated previously, we will first create ACLs, and then networks afterwards.

3. In the VPC list, select *My VPC*, then click *Network ACL Lists*. A list of default ACLs will be displayed.

The screenshot shows the AWS Management Console interface. At the top, there's a breadcrumb trail: Home / VPC. Below this is a table of VPCs. The first row is highlighted with a red box around the name 'My VPC'. The table has columns: Name, State, Description, and CIDR. Below the table, it says 'Showing 1-1 of 1 items' and '20 / page'. Below the VPC list, there's a section for 'My VPC' with details like Status (Enabled), ID (93f7a901-cdbd-4056-901e-7b3716dd27cb), VPC Offering (Default VPC offering), and Zone (Poc-Lab). To the right of these details is a list of Network ACLs. The 'Network ACL Lists' link is highlighted with a red box. The list shows two items: 'default_allow' (Default Network ACL Allow All) and 'default_deny' (Default Network ACL Deny All). The 'Add Network ACL List' link is also visible at the top right of the list.

4. Add two new ACLs, one for Frontend and one for Backend. Click *Add Network ACL List*, and complete the form to create the *Frontend ACL List* as follow. Once done, click *OK*:

The screenshot shows the 'Add Network ACL list' button, which is highlighted with a red box. Below this, the form fields are shown: 'ACL List Name:' with the value 'ACL-FE' and 'Description:' with the value 'Frontend ACL List'. At the bottom of the form, there are 'Cancel' and 'OK' buttons, with the 'OK' button highlighted with a red box.

5. Click [Add Network ACL List](#) again to create the [Backend ACL List](#), and follow the same process as before.

[Add Network ACL list](#)

ACL List Name: *ACL-BE*
 Description: *Backend ACL List*

Add ACL List

×

* ACL List Name :

ACL-BE

* Description :

Backend ACL List

Cancel

OK

6. Click [ACL-FE](#), select [ACL list Rules](#) to add an rule that denies all incoming traffic from the backend tier. Click [Add ACL](#) and complete the form as specified, and then click [OK](#).

Details

Networks

Public IP addresses

Network ACL lists

+

Add Network ACL list

Name	Description
ACL-BE	Backend ACL List
ACL-FE	Frontend ACL List
default_allow	Default Network ACL Allow All

Details

1 [ACL list rules](#)

2

+

Add ACL

↓

Export ACLs

Search

🔍

#Rule: *1*
 CIDR List: *192.168.0.128/25*
 Action: *Deny*
 Protocol: *All*
 Traffic Type: *Ingress*
 Description: *Deny all ingress traffic from backend tier.*

Add rule

#Rule: 1

CIDR list: 192.168.0.128/25

Action: Deny

Protocol: All

Traffic type: Ingress

Description: Deny all ingress traffic from backend tier.

Cancel OK

7. Return to ACL lists and click [ACL-BE](#), then click [ACL List Rules](#). Add a rule to allow incoming HTTP traffic (port 80) from the frontend tier by clicking [Add ACL](#) button and complete the form as specified, then click [OK](#).

Details	Add Network ACL list	
Networks	Name	Description
Public IP addresses	ACL-BE	Backend ACL List
Network ACL lists	ACL-FE	Frontend ACL List
	default_allow	Default Network ACL Allow All

Details
1 ACL list rules

2 + Add ACL
Export ACLs
Search

#Rule: 1
CIDR List: 192.168.0.0/25
Action: Allow
Protocol: TCP
Start Port: 80
End Port: 80
Traffic Type: Ingress
Description: Allow http ingress traffic from frontend tier.

Add rule [X]

#Rule: 1

CIDR list: 192.168.0.0/25

Action: Allow

Protocol: TCP

Start port: 80

End port: 80

Traffic type: Ingress

Description: Allow http ingress traffic from frontend tier.

Cancel OK

8. Click [Add ACL](#) again to deny any other ingress traffic from the frontend. Complete the form as specified, and then click [OK](#).

+ Add ACL

#Rule: 2

CIDR List: 192.168.0.0/25

Action: Deny

Protocol: All

Traffic Type: Ingress

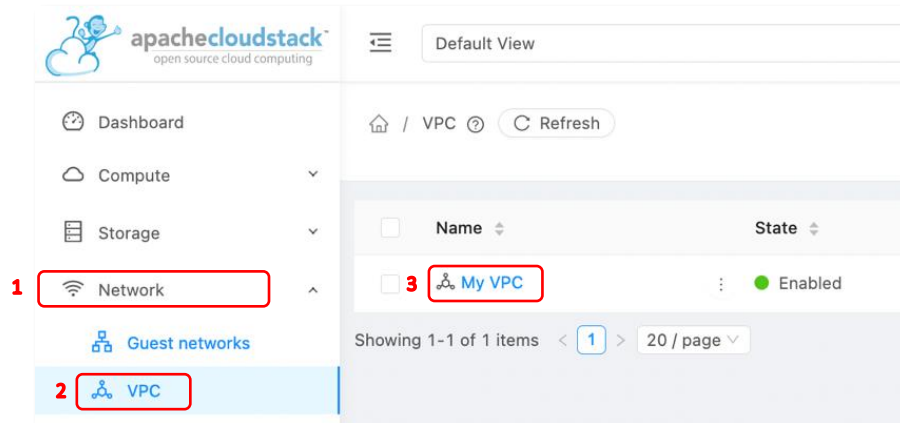
Description: Deny all ingress traffic from frontend tier.

Cancel OK

Creating Network Tiers

9. In the left navigation pane, go to [Network](#) > [VPC](#), then click [My VPC](#).





10. Click the [Networks](#) tab, then click [Add new Network Tier](#).



11. Complete the form as specified, then click [OK](#).

Name: *Frontend Tier*
 Network Offering: *Offering for Isolated Vpc networks with Source Nat service enabled*
 Gateway: *192.168.0.1*
 Netmask: *255.255.255.128*
 External Id: Blank
 ACL: *ACL-FE*

12. Click the [Networks](#) tab, then click [Add new Network Tier](#) again.

Name: *Backend Tier*
 Network Offering: *Offering for Isolated Vpc networks with Internal LB support*
 Gateway: *192.168.0.129*
 Netmask: *255.255.255.128*
 External Id: Blank
 ACL: *ACL-BE*

Add new Network Tier

* Name ⓘ

Backend Tier

* Network offering ⓘ

Offering for Isolated Vpc networks with Internal LB support

* Gateway ⓘ

192.168.0.129

* Netmask ⓘ

255.255.255.128

External Id ⓘ

ID of the Network in an external system.

* ACL ⓘ

ACL-BE (Backend ACL List)

Cancel

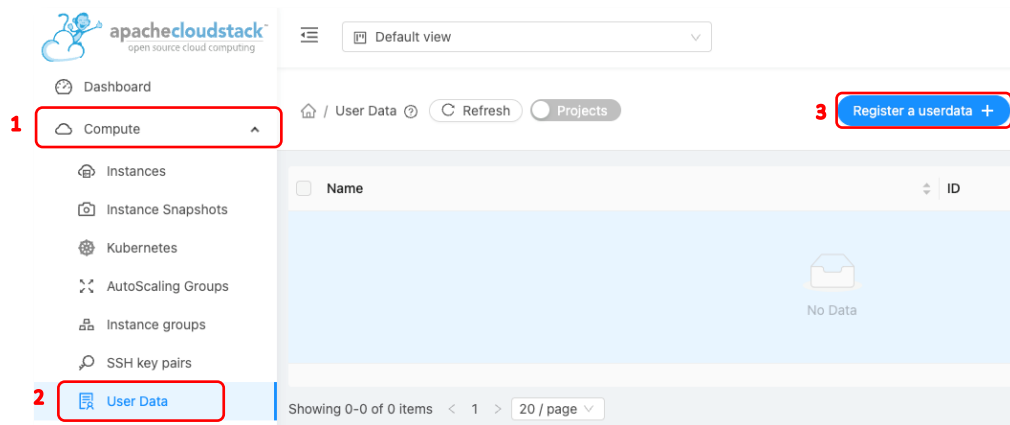
OK

Registering a Managed UserData Script for the Backend Instances

Note

The managed UserData script for the backend Instances automates the configuration of the Apache web server to run CGI scripts in Python. It enables the CGI module and updates the Apache configuration to allow CGI script execution in the `/var/www/html/` directory. The default file to be served is changed to `index.py`, which is set to be treated as a CGI script. The `index.py` script prints the Instance hostname in response to frontend requests.

13. In the left navigation pane, go to *Compute > User Data*, then click *Register a userdata*.



14. Complete the form as specified, then click **OK**.

Name: *backend-webserver*

Userdata: *## template: jinja*
#cloud-config
package_update: true
packages:
- apache2
runcmd:
- a2enmod cgid
- sed -i '/<Directory /var/www/>/!b;n;c\ Options Indexes FollowSymlinks
ExecCGI' /etc/apache2/apache2.conf
- sed -i 's|<Directory /var/www/>|<Directory /var/www/html>|'
/etc/apache2/apache2.conf
- sed -i 's/DirectoryIndex ./DirectoryIndex index.py/' /etc/apache2/mods-*
available/dir.conf
- sed -i 's|#AddHandler cgi-script .cgi|AddHandler cgi-script .py|'
/etc/apache2/mods-available/mime.conf
- rm -rf /var/www/html/index.html
- |
cat << 'EOF' > /var/www/html/index.py
#!/usr/bin/env python3
import socket
print("Content-type: text/plain\n")
print(socket.gethostname())
EOF
- chmod 705 /var/www/html/index.py
- systemctl enable --now apache2.service
- systemctl restart apache2.service

Base64 encoded: No

Userdata parameters: Blank

Register a userdata

Please fill in the following data to register a User data.

* Name

backend-webserver

* Userdata

```
## template: jinja
#cloud-config
package_update: true
packages:
- apache2
runcmd:
- a2enmod cgid
- sed -i '/<Directory /var/www/>/!b;n;c\ Options Indexes
```

Base64 encoded

☐

Userdata parameters

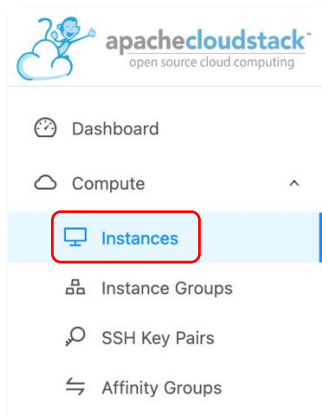
comma separated list of variables declared in userdata content

Cancel OK

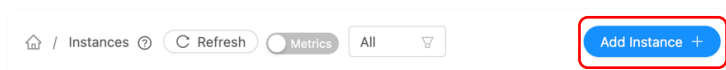


Creating Backend Instances

15. In the left navigation pane, go to [Compute](#), then select [Instances](#).



16. Click [Add Instance](#).



Note

To create each of the three backend Instances, modify only the *Name* field as follows: *backend-Instance-01*, *backend-Instance-02*, *backend-Instance-03*.

17. Complete the forms for each Instance (*backend-Instance-01*, *backend-Instance-02*, *backend-Instance-03*) as specified, then click [Launch Instance](#):

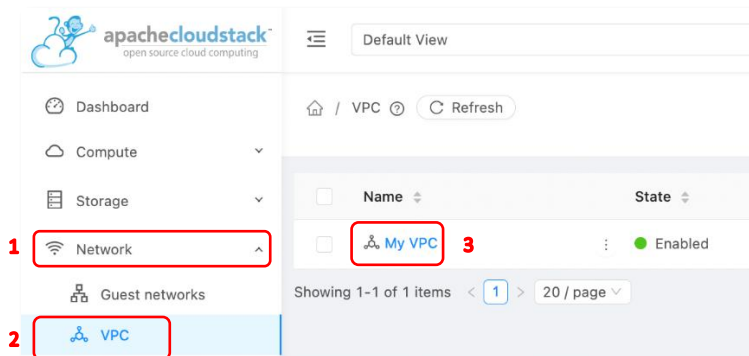
Zone: **poc-zone**
 Template: **Debian 11**
 Compute Offering: **Small Instance**
 Networks:
 - Backend Tier: **Yes**
 - Frontend Tier: **No**
 Advanced Mode: **Yes**
 - Stored Userdata: **backend-webserver**
 Name: **backend-Instance-0[1,2,3]**
 Group: **None**
 Keyboard Language: **None**
 Start Instance: **Yes**

Creating and configuring the Internal Load Balancer

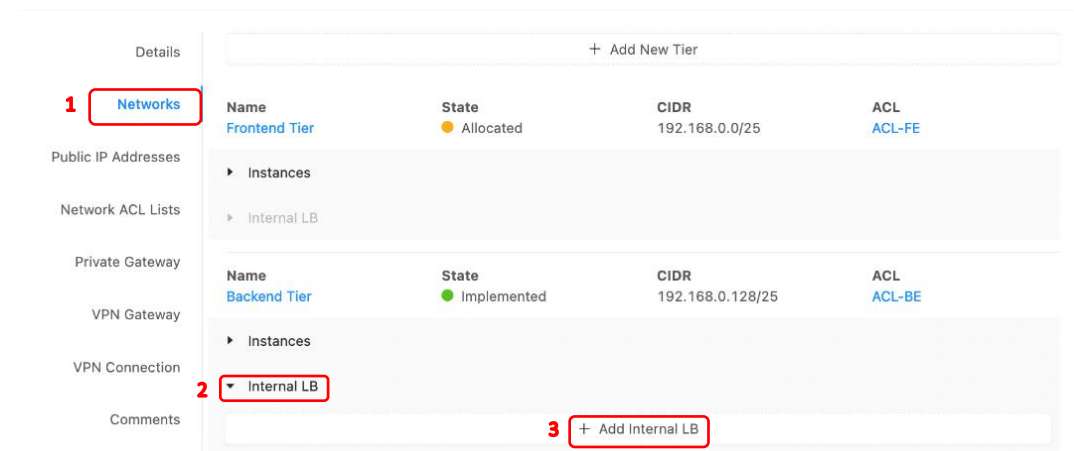
Note

The Internal Load Balancer distributes incoming requests from the Frontend Instances to the Backend Instances. As previously mentioned, each Backend Instance will respond with its hostname, allowing for load balancing in round-robin mode to ensure that requests are handled by different Instances.

18. In the left navigation pane, go to [Network](#), select [VPC](#), then select [My VPC](#).

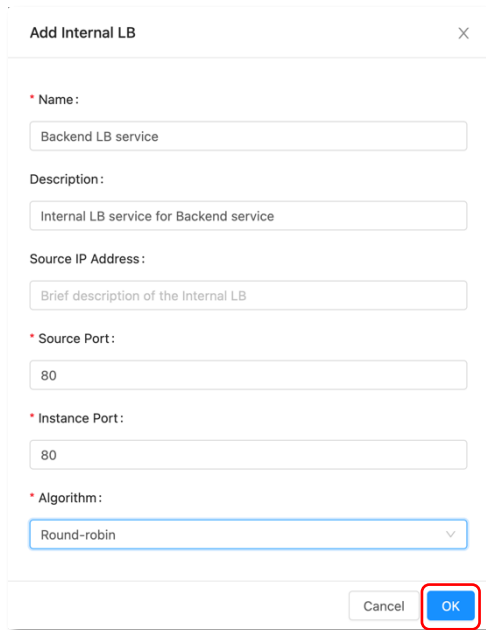


19. Click the [Networks](#) tab, expand the [Internal LB](#) menu, and then click [Add Internal LB](#).



20. Complete the form as follows, then click [OK](#).

Name: *Backend LB service*
Description: *Internal LB service for Backend service*
Source IP Address: Blank
Source Port: *80*
Instance Port: *80*
Algorithm: *Round-robin*



Add Internal LB

* Name:
Backend LB service

Description:
Internal LB service for Backend service

Source IP Address:
Brief description of the Internal LB

* Source Port:
80

* Instance Port:
80

* Algorithm:
Round-robin

Cancel OK

21. Copy the *Source IP Address*; it will be used later.

Name	State	CIDR	ACL
Backend Tier	Implemented	192.168.0.128/25	ACL-BE
Instances			
Internal LB			
+ Add Internal LB			
Name	Source IP Address	Algorithm	Account
Backend LB service	192.168.0.168	roundrobin	poc-user-account

22. Select the *Backend LB Service*.

Name	State	CIDR	ACL
Backend Tier	Implemented	192.168.0.128/25	ACL-BE
Instances			
Internal LB			
+ Add Internal LB			
Name	Source IP Address	Algorithm	Account
Backend LB service	192.168.0.168	roundrobin	poc-user-account

23. Click the *Assigned Instances* tab, then click *Assign Instance*.

Internal LB / Backend LB service Refresh

Backend LB service

Details **1** Assigned Instances Events

Name IP Address

24. Select all backend Instances, then click *Ok*.

Name	State	Internal name	Display Name	IP Address	Account	Zone Name	Select
backend-instance-03	Running		backend-instance-03		poc-user-account-1	PoC-Lab	☒
backend-instance-02	Running		backend-instance-02		poc-user-account-1	PoC-Lab	☒
backend-instance-01	Running		backend-instance-01		poc-user-account-1	PoC-Lab	☒

Total 3 items < 1 > 10 / page

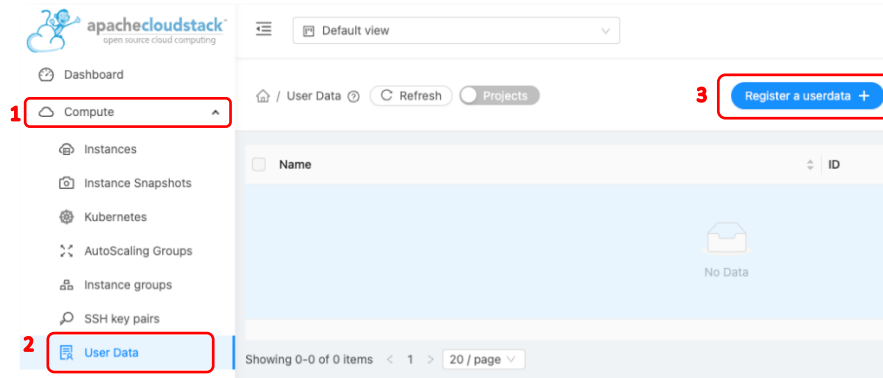
Cancel OK **2**

Registering a Managed UserData Script for the Frontend Instances

Note

The managed UserData script for the frontend Instances automates the configuration of the Apache web server to run CGI scripts in Python. It enables the CGI module and updates the Apache configuration to allow CGI script execution in the `/var/www/html/` directory. The default file to be served is changed to `index.py`, which sends a request to the backend Instances via the internal load balancer (using the `internal_lb` parameter) and displays the hostname of both the frontend and backend Instances.

25. In the left navigation pane, go to *Compute* and select *User Data*. Then click *Register a userdata*.



26. Complete the form as follows, then click **OK** to create the userdata.

Name: *frontend-webserver*

Userdata: *## template: jinja*
#cloud-config
package_update: true
packages:
- apache2
runcommand:
- a2enmod cgid
- sed -i '/<Directory \var/www/>!/b;n;c\ Options Indexes
FollowSymLinks ExecCGI' /etc/apache2/apache2.conf
- sed -i 's|<Directory /var/www/>|<Directory /var/www/html>|'
/etc/apache2/apache2.conf
- sed -i 's/DirectoryIndex ./DirectoryIndex index.py/' /etc/apache2/mods-*
available/dir.conf
- sed -i 's|#AddHandler cgi-script .cgi|AddHandler cgi-script .py|'
/etc/apache2/mods-available/mime.conf
- rm -rf /var/www/html/index.html
- |
cat << EOF > /var/www/html/index.py
#!/usr/bin/env python3
import requests
import socket
h = socket.gethostname()
r = requests.get("http://{{ ds.meta_data.internal_lb }}/")
print("Content-type: text/html\n\n")
print("<html>\n<body>")
print("<meta http-equiv=\"refresh\" content=\"10\"/>")
print("<div style=\"width: 100%; font-size: 40px; font-weight: bold; text-align:
center;\n>")
print('%s - %s' % (h, r.text))
print("</div>\n</body>\n</html>")
EOF
- chmod 705 /var/www/html/index.py
- systemctl enable --now apache2.service
- systemctl restart apache2.service

Base64 encoded: *No*

Userdata parameters: *internal_lb*



Register a userdata ?

×

Please fill in the following data to register a User data.

* Name ⓘ

frontend-webserver

* Userdata ⓘ

```
## template: jinja
#cloud-config
package_update: true
packages:
  - apache2
runcmd:
  - a2enmod cgi
  - sed -i '/<Directory \var/www/>/{b;n;c\ Options Indexes
FollowSymLinks ExecCGI' /etc/apache2/apache2.conf
```

Base64 encoded

☐

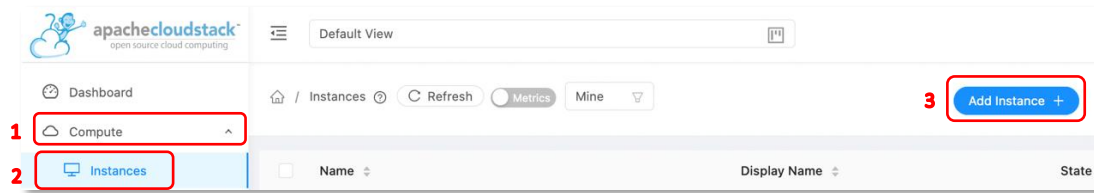
Userdata parameters ⓘ

internal_lb ×

Cancel OK

Creating Frontend Instances

27. In the left navigation pane, go to *Compute* and select *Instances*. then click *Add Instance*.



Note

When completing the Instances form, set the *internal_lb* key to the value copied from step 21. Additionally, update the *Name* field as follows: *frontend-Instance-01*, *frontend-Instance-02*.

28. Complete the forms for *frontend-Instance-01* and *frontend-Instance-02*, ensuring the *internal_lb* value is set correctly. Then, click *Launch Instance*. The *internal_lb* setting is crucial for enabling proper load balancing across the backend tier Instances behind the Internal Load Balancer.

Zone: *poc-zone*
 Template/ISO: *Debian 11*
 Compute Offering: *Small Instance*
 Networks:
 - Backend Tier: No
 - Frontend Tier: *Yes*
 Advanced Mode: *Yes*
 - Stored Userdata: *backend-webserver*
 internal_lb: *192.168.0.xxx*
 Name: *frontend-Instance-0[1,2]*
 Group: None
 Keyboard Language: None
 Start Instance: *Yes*

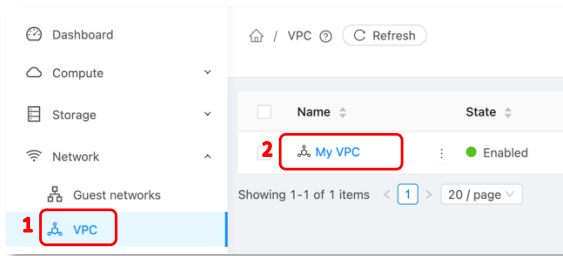
The screenshot shows the 'Stored Userdata' form in Apache CloudStack. It has a 'Manual Userdata entry' tab. Below the tab is a table with columns 'Userdata', 'Account', and 'Domain'. The table contains three entries: 'No thanks', 'backend-webserver', and 'frontend-webserver'. The 'frontend-webserver' entry is selected. Below the table is a form to add a new entry with fields for 'Key' and 'Value'. The 'internal_lb' key is entered, and the value '192.168.0.168' is entered in the 'Value' field.

Userdata	Account	Domain
☐ No thanks	-	-
☐ backend-webserver	poc-user-account	POC
☒ frontend-webserver	poc-user-account	POC
☐ webserver	poc-user-account	POC

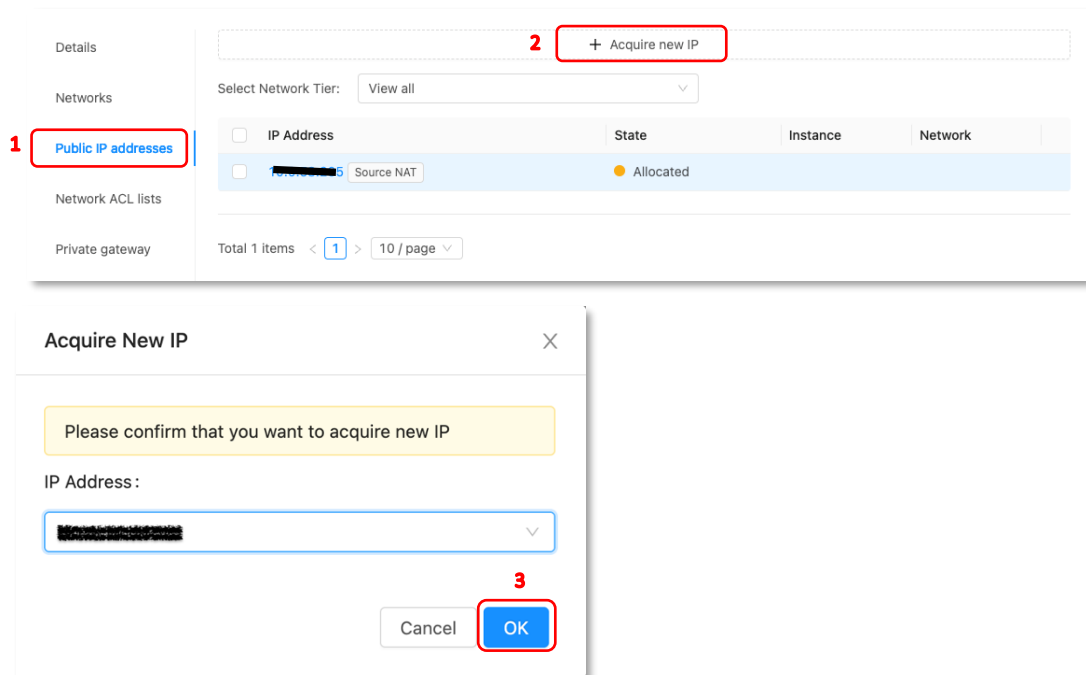
Key	Value
internal_lb	192.168.0.168

Creating and Configuring the External Load Balancer for Frontend Tier

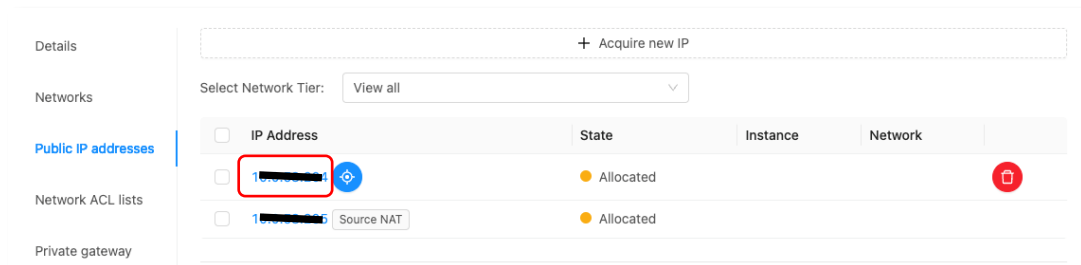
29. In the left navigation pane, go to *Network* > *VPC*, and then click *My VPC*.



30. Select the *Public IP Address* tab, then click *Acquire New IP*. Choose one IP Address from the list, then click *OK*.



31. Click on the acquired *IP address*.



32. Copy the *IP Address*; it will be used later.

Details Port forwarding Load balancing Events Comments

IP Address
10.0.58.204

33. Click the *Load Balancing* tab, complete the form as specified, and then click *Add*.

Name: *www*
 Public port: *80*
 Private port: *80*
 CIDR list: Blank
 Algorithm: *Round-robin*
 Protocol: *TCP*
 AutoScale: No

Details Port forwarding Load balancing Events Comments

* Name: www * Public port: 80 * Private port: 80

CIDR list: Algorithm: Round-robin Protocol: TCP AutoScale: No Add Instances: Add

34. Select the tier *Frontend Tier*, choose all Instances from the list, and then click *OK*.

Add Instances

Select Network Tier: Frontend Tier 1

Name	State	Display name	Account	Zone	Select
frontend-instance-01 192.168.0.99 (Primary) x	Running	frontend-instance-01	poc-user-account	Poc Zone	2 ☒
frontend-instance-02 192.168.0.14 (Primary) x	Running	frontend-instance-02	poc-user-account	Poc Zone	☒

Total 2 Items < 1 > 10 / page

Cancel OK 3

Accessing the Service within the VPC

35. Open a new tab in your browser, and paste the *IP Address* copied in the *step #32*.

Note

The web page will refresh automatically every 10 seconds. Each time the page reloads, you will see both the frontend and backend hostnames change. This demonstrates the load balancing in action, as the frontend Instance connects to different backend Instances via the internal load balancer, displaying the hostname of each Instance handling the request.

AutoScale Instance Group



Overview

Apache CloudStack provides an Autoscaling feature that enables automatic scaling of Instances in response to changes in demand. This is particularly useful for environments where workloads fluctuate, allowing resources to be dynamically adjusted based on predefined conditions or thresholds. Autoscaling is commonly used in both private and public cloud environments, ensuring that the necessary compute resources are available during peak times while minimizing costs during low usage periods.

The Autoscaling functionality in CloudStack integrates with the Virtual Router, which acts as a load balancer for distributing incoming traffic across multiple Instances. Based on pre-configured metrics, CloudStack will automatically increase (scale-out) or decrease (scale-in) the number of Instances to meet the demand. This provides a seamless way to manage resources efficiently without manual intervention.

Autoscaling relies on performance counters to monitor the state of the Instances. The following key metrics can be used to trigger scaling actions:

- **Instance CPU:** Average percentage of CPU utilization per Instance.
- **Instance Memory:** Average percentage of memory utilization per Instance.
- **Public Network (Receive):** Mbps received per Instance.
- **Public Network (Transmit):** Mbps transmitted per Instance.
- **Load Balancer:** Average number of connections per Instance.

In a typical setup, an Autoscaling Group defines the rules and conditions for scaling, including the minimum and maximum number of Instances, health checks, and the metrics to monitor. The Autoscaling Group ensures that the appropriate number of Instances are always running to handle traffic, scaling up when necessary and removing unneeded Instances during quieter periods.

Topics covered

- Creating Load Balancer with Autoscaling enabled
- Creating specific UserData
- Creating Autoscale Instance Group
- Simulating Autoscaling using Apache Benchmark

Accessing the Control Panel

1. To access the CloudStack UI, open the following URL in your web browser:

<http://10.0.32.10:8080/client>

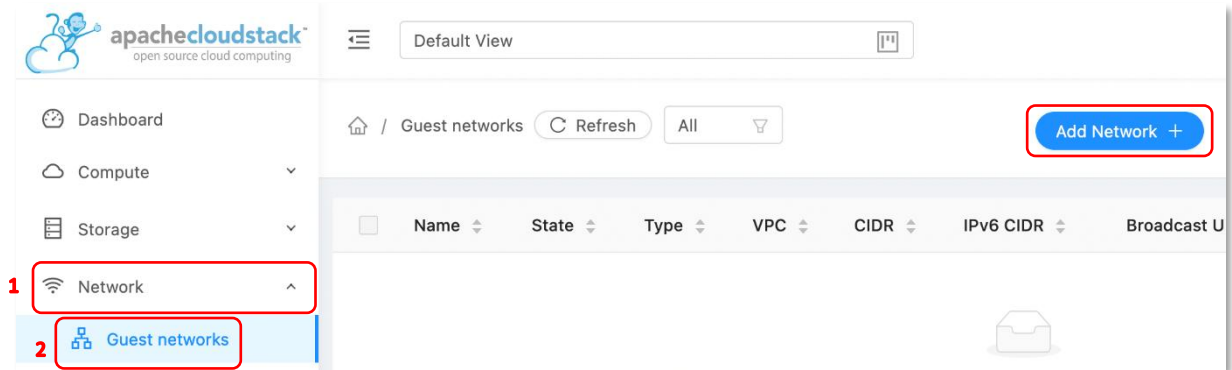
2. Login using the following credentials, then click [Login](#).

Username: *user-1*
Password: *password*
Domain: *poc*



Creating a Guest Network and Managing Egress Rules

3. In the left navigation pane, click [Networks](#) > [Guest networks](#), and then click [Add Network](#).



4. Select [Isolated](#), and fill in the form as follows and then click [OK](#):

Name:	autoscaling-network
Description:	Autoscaling Network
Zone:	Poc Zone
Network Offering:	Offering for Isolated networks with Source Nat service enabled
External Id:	Blank
Gateway:	Blank
Netmask:	Blank
DNS 1:	Blank
DNS 2:	Blank
Ipv4 address for the VR in this network:	Blank
Network Domain:	Blank

Add Network ⓘ

1 **Isolated** L2 Shared

* Name ⓘ
autoscaling-network

Description ⓘ
Autoscaling Network

* Zone ⓘ
Poc Zone

Network domain ⓘ
network domain

* Network offering ⓘ
Offering for Isolated networks with Source Nat service enabled

External Id ⓘ
ID of the network in an external system.

Gateway ⓘ
the gateway of the network. Required for shared networks and isolated networks when l...

Netmask ⓘ
the netmask of the network. Required for shared networks and isolated networks when ...

DNS 1 ⓘ DNS 2 ⓘ
the first IPv4 DNS for the network the second IPv4 DNS for the network

IPv4 address for the VR in this Network. ⓘ
IPv4 address to be assigned to the public interface of the network router. This address ...

Network domain ⓘ
network domain

Cancel OK 2

5. Click on [autoscaling-network](#) and then copy the CIDR address.

Home / Guest Networks ⓘ Refresh All Projects Add Network +

☐	Name	State	Type	VPC	IPv4 CIDR
1	autoscaling-network	Allocated	Isolated		10.1.1.0/24

autoscaling-network
 Isolated

Status
Allocated

ID
789a2854-06bd-450d-9297-76f730ca63a0

Network offering
DefaultIsolatedNetworkOfferingWithSourceNatService

Zone

Details Egress rules Public IP addresses VNF appliances Network permissions

Name
autoscaling-network

ID
789a2854-06bd-450d-9297-76f730ca63a0

Type
Isolated

Traffic type
Guest

CIDR
2 10.1.1.0/24

6. Click [Egress Rules](#) and add a rule to allow Instances to access the internet as follows. Then click [Add](#).

Source CIDR: 10.1.1.0/24
 Destination CIDR: 0.0.0.0/0
 Protocol: All

7. Below, you can see the added egress rule.

Source CIDR	Destination CIDR	Protocol	ICMP type / Start port	ICMP code / End port	Actions
☐ 10.1.1.0/24	0.0.0.0/0	All	All	All	

Total 1 items < 1 > 10 / page

Creating a Load Balancer with Autoscaling

Note

Unlike a traditional Load Balancer where Instances need to be manually associated, when Autoscale is enabled, there is no need to manually add Instances. The Autoscaling Group automatically manages this process by adding or removing Instances based on the defined scaling policies, ensuring that the Load Balancer always has the appropriate number of Instances to handle incoming traffic.

8. Now, click on [Public IP addresses](#) tab and acquire 2 new IP addresses by clicking on [Acquire new IP](#) button.

9. The two Public IP addresses will be displayed, with one of them serving as the *Source NAT* and the other designated for the Load Balancer. Click on the second IP to proceed.

☐	IP Address	State	Instance	Network	
☐	10.0.0.1 Source NAT	Allocated		autoscaling-network	
☐	10.0.0.2	Allocated		autoscaling-network	

10. Now, click on *Firewall* tab and enable incoming http connections completing the form as follows and then, click *Add*.

Source CIDR: Blank
 Protocol: TCP
 Start port: 80
 End port: 80

Details	Firewall	Port forwarding	Load balancing	Events	Comments
Source CIDR	Protocol	Start port	End port		
	TCP	80	80	Add	

11. The firewall rule will be listed as bellow.

☐	Source CIDR	Protocol	Start port/ICMP type	End port/ICMP code	State	Actions
☐	0.0.0.0/0	TCP	80	80	Active	

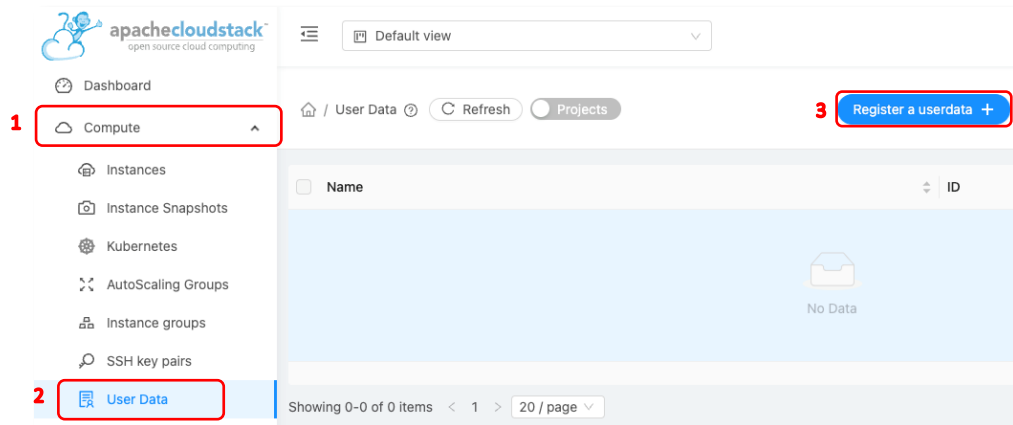
12. Click on *Load balancing* tab and complete the form as follows and then, click *Add*:

Name: my-autoscaling-app
 Public port: 80
 Private port: 80
 CIDR list: Blank
 Algorithm: Round-robin
 Protocol: TCP
 AutoScale: Yes

Details	Firewall	Port forwarding	Load balancing	Events	Comments
Name my-autoscaling-app Public port 80 Private port 80 CIDR list Algorithm Round-robin Protocol TCP AutoScale Yes Add Add					

Registering a Managed UserData Script for the Autoscaling

13. In the left navigation pane, go to *Compute > User Data*, then click *Register a userdata*.



14. Complete the form as specified, then click *OK*.

Name: *autoscaling-webserver-userdata*

Userdata: *#cloud-config*
package_update: true
packages:
- apache2
runcmd:
- a2enmod cgid
- sed -i '<Directory /var/www/>|!b;n;c\ Options Indexes FollowSymLinks
ExecCGI' /etc/apache2/apache2.conf
- sed -i 's|<Directory /var/www/>|<Directory /var/www/html>|'
/etc/apache2/apache2.conf
- sed -i 's|DirectoryIndex .|DirectoryIndex index.py|' /etc/apache2/mods-*
available/dir.conf
- sed -i 's|#AddHandler cgi-script .cgi|AddHandler cgi-script .py|' /etc/apache2/mods-
available/mime.conf
- rm -rf /var/www/html/index.html
- |
cat << EOF > /var/www/html/index.py
#!/usr/bin/env python3
import socket
import time
time.sleep(5)
print('Content-type: text/html\n\n')
print('<h1><p style="text-align: center;">Apache CloudStack Autoscaling
Demo</p></h1>')
print('<h2><p style="text-align: center;"> Instance:
{</p></h2>'.format(socket.gethostname()))
EOF
- chmod 705 /var/www/html/index.py
- systemctl enable --now apache2.service
- systemctl restart apache2.service

Base64 encoded: No
 Userdata parameters: Blank

Register a userdata ?

×

Please fill in the following data to register a User data.

* Name ⓘ

* Userdata ⓘ

```
## template: jinja
#cloud-config
package_update: true
package_upgrade: true
packages:
  - apache2
runcmd:
```

Base64 encoded

☐

Userdata parameters ⓘ

Cancel

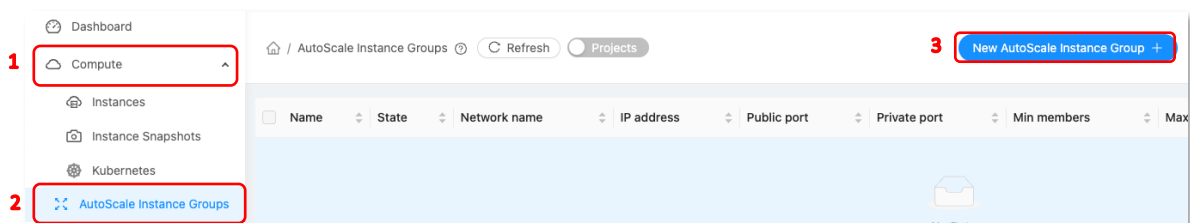
OK

Note

The managed UserData script for the Autoscaling Instances Group automates the configuration of the Apache web server to run CGI scripts in Python. It enables the CGI module and updates the Apache configuration to allow CGI script execution in the `/var/www/html/` directory. The default file to be served is changed to `index.py`, which is set to be treated as a CGI script. The `index.py` script fetches the hostname of backend instances and displays it alongside the frontend hostname. A `sleep 5` command is included in the script to simulate a slight delay, which is useful for testing how the Autoscaling system handles load and latency under real-world conditions, allowing users to observe the scaling behaviour in action.

Creating an Autoscaling Instance Group

15. In the left navigation pane, go to *Compute > AutoScale Instance Groups*, then click *New AutoScale Instance Group*.



16. The form *New AutoScale Instance Group* will be shown. First, select a *Zone* where the Autoscaling Group will be created.

1 Select a zone

A zone typically corresponds to a single datacenter. Multiple zones help make the cloud more reliable by providing physical isolation and redundancy.

* Zone

☒ Poc Zone

17. Next, select the *Debian 11* Template.

2 Select a Template

Templates

Search

OS image that can be used to boot instances.

Featured Community My Templates Shared

☒ Debian 11

Override root disk size ☐

Total 1 items < 1 > 10 / page

Next, select a *Compute offering*.

3 Compute offering

Search

Compute offering	CPU	Memory
☒ Small Instance	1 CPU x 0.50 Ghz	512 MB
☐ Medium Instance	1 CPU x 1.00 Ghz	1024 MB

Override root disk offering ☐

Total 2 items < 1 > 10 / page

18. Select the Network *autoscaling-network*.

5 Select Network
Please choose at least one Network for Instances in the autoscale Instance group. The default Network must be an Isolated Network or VPC Network Tier which supports Instance AutoScaling and has load balancing rules.

[Create new Network](#)

☒ Networks	Guest type	VPC	Supports auto scaling
☒  autoscaling-network	Isolated		Yes

Total 1 items < 1 > 10 / page

Please configure the IP address and mac address of networks if needed. Please select a network as the default network.

Network	IP address	MAC address
☒ autoscaling-network CIDR: 10.1.1.0/24		

19. Select the Load Balancing *my-autoscaling-app*.

6 Load balancing
Please select a load balancer rule for your AutoScale Instance group.

Name	IP address	Public port	Private port
☒ my-autoscaling-app	10.1.55.185	80	80

Total 0 items < 1 > 10 / page

Note

In this simulation example, we will create a ScaleUp and ScaleDown Policies based on the number of load balancer connections per Instance. The values used are reduced and do not reflect a real-world scenario. The purpose here is to demonstrate the concept and illustrate how the scaling mechanism works. This simplified approach focuses on applying the principles of Autoscaling and understanding its functionality rather than replicating production-level scaling conditions.

20. Complete the form to create a *Scale Up Policy*, then click *Add condition*.

Name: ScaleUpPolicy-0
 Duration (in sec): 20
 Quiet time (in sec): 20
 Counter: Load Balancer - average connections per vm
 Operator: Greater than
 Threshold: 1

7 ScaleUp policies
 Please add at least a ScaleUp policy. The AutoScale Instance group will be scaled up when all conditions in a ScaleUp policy are matched. ScaleUp policies will be checked before ScaleDown policies.

ScaleUp policy: ScaleUpPolicy-0 + Add policy Remove policy

* Name ⓘ * Duration (in sec) ⓘ Quiet time (in sec) ⓘ

ScaleUpPolicy-0 20 20

* Counter ⓘ * Operator ⓘ * Threshold ⓘ **Action**

Load Balancer - average connections per vm Greater than 1 + Add condition

Note

The ScaleUp Policy defines the conditions under which an AutoScale Instance Group will automatically scale up, meaning new Instances will be added to handle increased demand. When creating an AutoScale Instance Group, at least one ScaleUp policy is required. The scaling process is triggered when all conditions in the ScaleUp policy are met.

21. Next, complete the form to create a *Scale Down Policy*, then click *Add condition*.

Name: ScaleDownPolicy-0
 Duration (in sec): 20
 Quiet time (in sec): 20
 Counter: Load Balancer - average connections per vm
 Operator: Less than
 Threshold: 1

8 ScaleDown policies
 Please add at least a ScaleDown policy. The AutoScale Instance group will be scaled down when all conditions in a ScaleDown policy are matched. ScaleDown policies will be checked after ScaleUp policies.

ScaleDown policy: ScaleDownPolicy-0 + Add policy Remove policy

* Name ⓘ * Duration (in sec) ⓘ Quiet time (in sec) ⓘ

ScaleDownPolicy-0 20 20

* Counter ⓘ * Operator ⓘ * Threshold ⓘ **Action**

Load Balancer - average connections per vm Less than 1 + Add condition

Note

The ScaleDown Policy defines the conditions under which an AutoScale Instance Group will automatically scale down, meaning Instances will be removed when demand decreases. Unlike the ScaleUp Policy, the ScaleDown policy ensures that resources are reduced when conditions meet specified thresholds. The scaling down process is triggered when all conditions in the ScaleDown policy are satisfied, and the system will decrease the number of Instances to optimize resource usage during lower demand periods.

22. Now, enable *Advanced Mode*, then select the *autoscaling-webserver-userdata* in the *Userdata* section.

The screenshot shows the 'Advanced mode' settings for an AutoScaling Group. The 'Show advanced settings' toggle is turned on. Under the 'Stored Userdata' tab, there is a search bar and a table with columns: 'Userdata', 'Account', and 'Domain'. The table has two rows: 'No thanks' and 'autoscaling-webserver-userdata'. The 'autoscaling-webserver-userdata' row is selected, and its 'Account' is 'poc-user-account' and 'Domain' is 'POC'. A red box highlights the 'autoscaling-webserver-userdata' row.

Userdata	Account	Domain
☐ No thanks	-	-
☒ autoscaling-webserver-userdata	poc-user-account	POC

23. To finish the *AutoScale Instance Group* creation, complete the fields in the *Details* section and then click *Create*.

Name: **webserver-autoscaling-group**

Expunge Instance grace period (in sec): **20**

Max members: **3**

Min members: **1**

Polling interval (in sec): **20**

10 Details

* Name ⓘ
webserver-autoscaling-group

* Expunge Instance grace period (in sec) ⓘ
20

* Max members ⓘ
3

* Min members ⓘ
1

* Polling interval (in sec) ⓘ
20

Cancel Create

Note

The *Expunge Instance Grace Period* (20 seconds) defines the time an instance will remain in the system before being permanently removed after it is marked for expunging. The *Max Members* is set to 3, meaning the *AutoScaling Group* can have up to 3 Instances running simultaneously, while the *Min Members* is set to 1, ensuring at least one Instance is always running. The *Polling Interval* (20 seconds) specifies how frequently the system checks the scaling conditions, determining whether to scale up or down based on the current load and metrics.

24. The *webserver-autoscaling* group is now listed on the *AutoScale Instance Group* page. Click on it to view or manage its details.

Dashboard

Compute

- Instances
- Instance Snapshots
- Kubernetes
- AutoScale Instance Gr...**
- Instance groups

Home / AutoScale Instance Groups ⓘ Refresh Projects

☐	Name	State	Network name
☐	webserver-autoscaling	● ENABLED	autoscaling-network

Showing 1-1 of 1 items < 1 > 20 / page ▾

25. Next, click on the [Events](#) tab. Here, you can view all [SCALEUP](#) events related to the [AutoScale Instance Group](#).

Details AutoScale Instance Profile Load balancing rule ScaleUp policy ScaleDown policy Events Comments							
Level	Type	State	Username	Account	Domain	Created	
INFO	AUTOSCALEVMGROUP.CREATE	Scheduled	user-1	poc-user-account	POC	24 Oct 2024 12:22:51	
INFO	AUTOSCALEVMGROUP.CREATE	Started	user-1	poc-user-account	POC	24 Oct 2024 12:22:51	
INFO	AUTOSCALEVMGROUP.CREATE	Completed	user-1	poc-user-account	POC	24 Oct 2024 12:22:54	
INFO	AUTOSCALEVMGROUP.SCALEUP	Started	system	poc-user-account	POC	24 Oct 2024 12:23:11	
INFO	AUTOSCALEVMGROUP.SCALEUP	Completed	system	poc-user-account	POC	24 Oct 2024 12:23:18	

26. Related to the SCALEUP event, as defined in the [AutoScale Instance Group](#), an Instance will be created based on the minimum number of Instances specified.

The screenshot shows the Apache CloudStack web interface. On the left is a sidebar with navigation options: Dashboard, Compute, Instances (selected), Instance Snapshots, Kubernetes, AutoScale Instance Gr..., Instance groups, SSH key pairs, User Data, and Affinity groups. The main panel shows the 'Instances' view with a 'Default view' dropdown, a 'Refresh' button, and filters for 'Mine', 'Metrics', and 'Projects'. A table of instances is displayed with columns for Name and State. One instance, 'autoScaleVm-autoscaling-webserver-group-1-zmockz', is highlighted and shown as 'Running'. Below the table, it indicates 'Showing 1-1 of 1 items' and a pagination control for '20 / page'.

Simulating Autoscaling using Apache Benchmark

Note

To perform the web request simulation against the load balancer, ensure that [Apache Benchmark \(ab\)](#) is installed on your desktop. This tool is required to execute the task and simulate traffic towards the load balancer for testing the autoscaling behaviour.

27. To generate traffic in the load balancer, copy *the IP address* from the *webserver-autoscaling* Autoscaling Group in the *Details* page.

Details	AutoScale Instance Profile	Load balancing rule	ScaleUp policy	ScaleDown policy	Events	Comments
Name						
webserver-autoscaling						
ID						
30a6a154-13c6-4056-8014-b770b6f61209						
Account						
poc-user-account						
Domain						
POC						
Network name						
autoscaling-network						
Associated Network ID						
22ad1e86-9c46-47a9-bed5-af683e118d4d						
Load balancer ID						
65f61a8c-e2ec-44da-a006-0d9b7718144e						
Load balancer provider						
VirtualRouter						
IP address						
10.0.0.1						

28. Next, open a terminal on your desktop and run the following command, replacing *ip_address* with the *AutoScale Instance Group* IP address you copied earlier.

```
ab -n 200 -c 4 http://ip_address/
```

Note

The Apache Benchmark (`ab -n 200 -c 4 http://ip_address/`) command will send 200 total requests to the specified IP address, with 4 concurrent requests at a time. Given that the webpage has a 5-second delay (due to the `time.sleep(5)` in the Python script), the overall time for the load test will account for the processing of each request. Depending on network latency and server response times, this will help simulate load and observe the behaviour of the Autoscaling Group under stress, especially as it scales to meet demand.

29. As the command runs, return to the CloudStack panel, go to the Autoscaling Instance Group [Events](#) tab, and periodically click the [Refresh](#) button to monitor the autoscaling actions.

Home / AutoScale Instance Groups / autoscaling-webserver-group ? [Refresh](#)

Details AutoScale Instance Profile Load balancing rule ScaleUp policy ScaleDown policy **Events** Comments

Level	Type	State	Username	Account	Domain	Created
INFO	AUTOSCALEVMGROUP.CREATE	Scheduled	user-1	poc-user-account	POC	24 Oct 2024 12:22:51
INFO	AUTOSCALEVMGROUP.CREATE	Started	user-1	poc-user-account	POC	24 Oct 2024 12:22:51
INFO	AUTOSCALEVMGROUP.CREATE	Completed	user-1	poc-user-account	POC	24 Oct 2024 12:22:54
INFO	AUTOSCALEVMGROUP.SCALEUP	Started	system	poc-user-account	POC	24 Oct 2024 12:23:11
INFO	AUTOSCALEVMGROUP.SCALEUP	Completed	system	poc-user-account	POC	24 Oct 2024 12:23:18
INFO	AUTOSCALEVMGROUP.SCALEUP	Started	system	poc-user-account	POC	24 Oct 2024 12:31:23
INFO	AUTOSCALEVMGROUP.SCALEUP	Completed	system	poc-user-account	POC	24 Oct 2024 12:31:30

30. By accessing the Instances list, you will notice that new Instances are being created.

Instances

- Instance Snapshots
- Kubernetes
- AutoScale Instance Gr...
- Instance groups
- SSH key pairs
- User Data
- Affinity groups

☐ Name

☐ autoScaleVm-autoscaling-webserver-group-1-zmockz

☐ autoScaleVm-autoscaling-webserver-group-2-xrbrwn

Showing 1-2 of 2 items < 1 > 20 / page

31. As the Apache Benchmark command continues generating traffic to the load balancer endpoint, the SCALEUP trigger will keep scaling the number of Instances until the maximum limit defined in the *AutoScale Instance Group* is reached.

Details AutoScale Instance Profile Load balancing rule ScaleUp policy ScaleDown policy Events Comments							
Level	Type	State	Username	Account	Domain	Created	
INFO	AUTOSCALEVMGROUP.CREATE	Scheduled	user-1	poc-user-account	POC	24 Oct 2024 12:22:51	
INFO	AUTOSCALEVMGROUP.CREATE	Started	user-1	poc-user-account	POC	24 Oct 2024 12:22:51	
INFO	AUTOSCALEVMGROUP.CREATE	Completed	user-1	poc-user-account	POC	24 Oct 2024 12:22:54	
INFO	AUTOSCALEVMGROUP.SCALEUP	Started	system	poc-user-account	POC	24 Oct 2024 12:23:11	
INFO	AUTOSCALEVMGROUP.SCALEUP	Completed	system	poc-user-account	POC	24 Oct 2024 12:23:18	
INFO	AUTOSCALEVMGROUP.SCALEUP	Started	system	poc-user-account	POC	24 Oct 2024 12:31:23	
INFO	AUTOSCALEVMGROUP.SCALEUP	Completed	system	poc-user-account	POC	24 Oct 2024 12:31:30	
INFO	AUTOSCALEVMGROUP.SCALEUP	Started	system	poc-user-account	POC	24 Oct 2024 12:32:30	
INFO	AUTOSCALEVMGROUP.SCALEUP	Completed	system	poc-user-account	POC	24 Oct 2024 12:32:39	

Instances

Instance Snapshots

Kubernetes

AutoScale Instance Gr...

Instance groups

SSH key pairs

User Data

Affinity groups

☐ Name

☐ autoScaleVm-autoscaling-webserver-group-1-zmockz

☐ autoScaleVm-autoscaling-webserver-group-2-xrbrwn

☐ autoScaleVm-autoscaling-webserver-group-3-hhrtfi

Showing 1-3 of 3 items < 1 > 20 / page

32. When the Apache Benchmark command completes, the **SCALEDOWN** trigger will be executed, and the instances will be removed until the number of Instances returns to the minimum defined in the *AutoScale Instance Group*.

Details AutoScale Instance Profile Load balancing rule ScaleUp policy ScaleDown policy Events Comments							
Level	Type	State	Username	Account	Domain	Created	
INFO	AUTOSCALEVMGROUP.SCALEUP	Started	system	poc-user-account	POC	24 Oct 2024 12:23:11	
INFO	AUTOSCALEVMGROUP.SCALEUP	Completed	system	poc-user-account	POC	24 Oct 2024 12:23:18	
INFO	AUTOSCALEVMGROUP.SCALEUP	Started	system	poc-user-account	POC	24 Oct 2024 12:31:23	
INFO	AUTOSCALEVMGROUP.SCALEUP	Completed	system	poc-user-account	POC	24 Oct 2024 12:31:30	
INFO	AUTOSCALEVMGROUP.SCALEUP	Started	system	poc-user-account	POC	24 Oct 2024 12:32:30	
INFO	AUTOSCALEVMGROUP.SCALEUP	Completed	system	poc-user-account	POC	24 Oct 2024 12:32:39	
INFO	AUTOSCALEVMGROUP.SCALEDOWN	Started	system	poc-user-account	POC	24 Oct 2024 12:34:39	
INFO	AUTOSCALEVMGROUP.SCALEDOWN	Completed	system	poc-user-account	POC	24 Oct 2024 12:35:09	
INFO	AUTOSCALEVMGROUP.SCALEDOWN	Started	system	poc-user-account	POC	24 Oct 2024 12:36:09	
INFO	AUTOSCALEVMGROUP.SCALEDOWN	Completed	system	poc-user-account	POC	24 Oct 2024 12:36:39	


apachecloudstack
open source cloud computing

Dashboard
Compute
Instances
Instance Snapshots
Kubernetes
AutoScale Instance Gr...
Instance groups
SSH key pairs
User Data
Affinity groups

Default view

/ Instances [Refresh](#) [Mine](#) [Metrics](#) [Projects](#) [Add Instance +](#)

Name	State
 autoScaleVm-autoscaling-webserver-group-1-zmockz	Running

Showing 1-1 of 1 items [1](#) / 20 / page

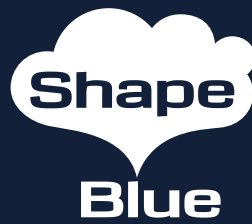
References and Resources



For additional information and support, you can explore the following resources:

- Apache CloudStack Official Website: Visit the [CloudStack website](#) to learn more about its features, updates, and community.
- Official Documentation: Access the [Apache CloudStack Documentation](#) for detailed guidance on configuring and managing your CloudStack environment.
- ShapeBlue Website: Discover more about CloudStack support, services, consultancy, and training at [ShapeBlue](#).
- Advisory Session with a Cloud Architect: Schedule a session to discuss your CloudStack architecture and deployment with a professional architect at ShapeBlue. Visit [Get an Advisory Session](#) for more details.





Need help with **Apache CloudStack**?

Get in Touch



cloudstack.apache.org/

About **Apache CloudStack**

Apache CloudStack is the leading open source cloud orchestration platform, in use by many of the world's largest public and private clouds. It is a multi-hypervisor, multi-tenant, high-availability Infrastructure as a Service cloud management platform. CloudStack is software that provides a cloud orchestration layer, giving automation of the creation, provisioning and configuration of IaaS components.

CloudStack turns an existing virtual infrastructure into a cloud-based infrastructure as a Service (IaaS) platform. The fact CloudStack leverages existing infrastructure means that the cost and time for an organisation to build a multi-tenant IaaS platform is greatly reduced.



@ info@shapeblue.com www.shapeblue.com

About **ShapeBlue**

ShapeBlue is the largest independent integrator of CloudStack technologies globally and are specialists in the design and implementation of IaaS cloud infrastructures for both private and public cloud implementations. We combine 100's of person-years of experience in designing and building complex network, storage and compute infrastructures with globally leading skills in Apache CloudStack.